

UNIVERSITETI “ISMAIL QEMALI” VLORE
FAKULTETI I SHKENCAVE TEKNIKE DHE NATYRORE



DEPARTAMENTI I SHKENCAVE KOMPJUTERIKE

PUNIM DIPLOME

MASTER I SHKENCAVE NË SHKENCA KOMPJUTERIKE

**TEMA: MONITORIMI I TEMPERATURËS DHE LAGËSHTISË NË KOHË REALE
ME SENSORË DHE PYTHON**

Punoi: Dionis Mulita

Pedagog Udhëheqës: Dr. Eljona Zana

VLORE 2026

DEKLARATË ORIGINALITETI

Unë, Dionis Mulita, deklaroj se ky punim diplome përfaqëson punën time origjinale dhe nuk është kopjuar nga burime të tjera pa cituar autorësinë. Të gjitha burimet e përdorura janë referuar sipas rregullave akademike. Përmbajtja e këtij punimi është rezultat i punës sime personale gjatë studimeve master në Shkenca Kompjuterike.

Nënshkrimi: _____

Dionis Mulita

MIRËNJOHJE

Dëshiroj të shpreh mirënjohjen time më të thellë për pedagogun tim udhëheqës, Dr. Eljona Zana, për udhëzimin, durimin dhe mbështetjen e saj të vazhdueshme gjatë gjithë kohës së punimit të kësaj teze. Këshillat dhe ekspertiza e saj kanë qenë të paçmueshme në çdo fazë të këtij projekti. Falënderoj gjithashtu stafin akademik të Departamentit të Shkencave Kompjuterike për mbështetjen dhe njohuritë që më kanë dhënë gjatë viteve të studimit. Një falënderim i veçantë shkon për familjen time për durimin dhe inkurajimin e tyre të vazhdueshëm. Pa mbështetjen e tyre, ky punim s'do të ishte i mundur.

ABSTRAKTI

Sensorët e lidhur në internet prodhojnë vazhdimisht të dhëna sot, por shpesh nuk kemi mjete të thjeshta për t'i analizuar ato. Kjo tezë trajton pyetjen praktike se si mund të ndërtohet një platformë e hapur dhe e integruar për monitorimin inteligjent të temperaturës dhe lagështisë duke përdorur ESP32 me DHT22, FastAPI dhe Python. Përfundimet kryesore tregojnë se ESP32 me DHT22 japin saktësi të krahasueshme me sensorin profesional Tuya TH06, IQR zbulon anomali me saktësi mbi 90%, dhe regresioni polinomial jep parashikime 24-orëshe më të sakta se ai linear. Platforma është e lehtë për t'u vendosur dhe zgjerohet lehtë.

Në panelin e kontrollit vendosa butona për zgjedhjen e intervalit kohorë (5m, 15m, 1h, 6h, 24h), duke lejuar përdoruesin të fokusohet në të dhënat më të fundit ose të shohë trende afatgjata. Kur zgjedh një interval, serveri kthen vetëm të dhënat nga ajo periudhë.

Kur përdoruesi kalon miun mbi një pikë të grafikut, shfaqet një tooltip, duke treguar vlerën e saktë të temperaturës ose lagështisë në atë moment. Boshti Y i temperaturës është vendosur nga -15°C në 75°C me hap 1°C, ndërsa boshti i lagështisë nga 0% në 100%. Boshti kohorë shfaq orën në formatin HH:mm:ss me dy pika ndarëse. Lartësia e grafikëve është rritur në 70% të lartësisë së dritares për pamje më të mirë.

Fjalë kyçe: IoT, ESP32, FastAPI, Python, Monitorim Temperature, DHT22, Zbulim Anomalish, Random Forest, Regresion Polinomial, Chart.js, Tuya, Telegram

ABSTRACT

Internet-connected sensors generate enormous streams of data every second. This thesis addresses the practical question of how to collect, analyze, and act upon this data using open and accessible technologies. The developed platform consists of a FastAPI Python server that receives real-time data from ESP32 sensors with DHT22, analyzes them for anomalies using the Interquartile Range (IQR) method, and predicts future trends through polynomial regression and Random Forest. Data is stored in memory for speed and persisted in JSON files for durability. The dashboard is a single-page application (SPA) with Chart.js displaying real-time temperature and humidity charts via WebSocket. Notifications are sent to Telegram when anomalies are detected. A built-in Tuya bridge enables connection with commercial Tuya TH06 devices without complex configuration. Results show that the platform detects anomalies with high accuracy and predicts future values with a mean error below 0.5°C.

Keywords: IoT, ESP32, FastAPI, Python, Temperature Monitoring, DHT22, Anomaly Detection, Random Forest, Polynomial Regression, Chart.js, Tuya, Telegram

PËRMBAJTJA

Kapitulli I – Hyrja	10
1.1. Konteksti dhe rëndësia e studimit.....	10
1.2 Përdorimi i ESP32 dhe Python në IoT.....	10
1.3 Qëllimi i studimit.....	11
1.4 Struktura e punimit.....	12
1.5 Përmbledhje.....	12
Kapitulli II – Sfondi Teorik	13
2.1 Koncepti i Internet of Things (IoT) dhe arkitektura me katër shtresa ..	13
2.2 Sensorët e temperaturës dhe lagështisë: DHT22, specifikimet dhe krahasimi ..	16
2.3 Platformat harduerike: ESP32 dhe Arduino	19
2.4 Bibliotekat Python: FastAPI, scikit-learn, pandas, numpy, Chart.js ...	21
2.5 Protokollet e komunikimit: HTTP REST dhe WebSocket	27
2.6 Mënyrat e ruajtjes së të dhënave: JSON në memorie dhe skedarë ...	30
2.7 Përmbledhje	30
Kapitulli III – Formulimi i problemit dhe qëllimet e hulumtimit .	31
3.1 Problemi: Mungesa e një platforme të thjeshtë, të hapur dhe të integruar për monitorim inteligjent IoT .	31
3.2 Pyetjet e hulumtimit: ndërtimi i platformës, analiza statistikore, parashikimi, krahasimi me pajisje profesionale .	32
3.3 Hipotezat: saktësia e DHT22, efikasiteti i IQR për anomalitë, vlefshmëria e parashikimeve	32
3.4 Qëllimet: ndërtimi i sistemit, analiza e të dhënave, parashikimi, krahasimi me Tuya TH06	33
3.5 Objektivat specifike: arkitektura, sensorët, serveri, dashboard-i, analiza ML, testimi	35
3.6 Përmbledhje	34
Kapitulli IV – Metodologjia dhe teknikat	36
4.1 Pajisjet: ESP32 DevKit V1, DHT22, Tuya TH06	36
4.2 Mjedis zhvillimor: Python 3.11, FastAPI, Windows	36
4.3 Procesi i zhvillimit: analiza e kërkesave, dizajn, implementim, testim ...	37
4.4 Komunikimi hibrid: REST për pajisjet, WebSocket për dashboard-in ...	37
4.5 Ruajtja dhe vizualizimi: JSON, Chart.js me grafikë të dyfishtë ..	37
4.6 Teknikat AI: IQR për zbulimin e anomalive, Polynomial Regression për parashikim.....	42

4.7 Testimi: injektimi i anomalive, krahasimi me Tuya, matja e saktësisë ...	42
4.8 Shëmbuj kodë	43
4.9 Përmbledhje	45
Kapitulli V – Rezultatet e hulumtimit	46
5.1 Ndërtimi i sistemit: 8 profile sensorësh (HIGH-TEMP, LOW-TEMP, CONSTANT, DRIFT, COLD-ROOM, HOT-ZONE, WIDE-SWING, STABLE-HUMID) ...	47
5.2 Mbledhja dhe ruajtja: ~17,280 lexime/pajisje në 24 orë	48
5.3 Statistikat dhe zbulimi i anomalive: IQR me dritare rrotulluese 50 lexime ...	48
5.4 Parashikimi 24-orësh me breza besimi 95% (Polynomial Regression) ...	49
5.5 Krahasimi me Tuya TH06: devijim mesatar 0.3°C, brenda specifikimeve $\pm 0.5^{\circ}\text{C}$	50
5.6 Rritja e jitter-it në $\pm 0.5^{\circ}\text{C}$ për pamje më natyrore të të dhënave ...	52
5.7 Diskutimi i rezultateve	52
5.8 Përmbledhje	52
Kapitulli VI – Diskutimi dhe përmirësimet	53
6.1 Avantazhet e Sistemit	53
6.2 Kufizimet: ndërfaqe bazike, ruajtje në memorie, mundësi autentifikimi, pa multi-user	54
6.3 Përmirësimet: bazë të dhënash SQL, autentifikimi JWT/OAuth2, aplikacion mobil, enkriptim TLS ...	54
6.4 Përmbledhje	54
Kapitulli VII – Përfundimet dhe rekomandimet	56
7.1 Përfundimet: platforma IoT+AI funksionale, DHT22 i saktë, Random Forest për parashikime të besueshme ...	56
7.2 Rekomandimet: migrimi në SQL, autentifikimi, aplikacion mobil, sensorë shtesë.....	56
7.3 Përfundim i përgjithshëm	56
7.4 Përmbledhje	56
Kapitulli VIII – Zhvillimi i Punës Praktike	57
8.1 Hapat e konfigurimit të mjedisit dhe regjistrimi i sensorit fizik në dashboard ...	57
8.2 Programimi i ESP32 me Arduino IDE dhe dërgimi i të dhënave me POST ...	58
8.3 Ndërtimi i serverit FastAPI me endpoint-et REST dhe WebSocket ...	59
8.4 Dashboard-i interaktiv me Chart.js dhe panel kontrolli	60
8.5 Zbulimi i anomalive me IQR dhe Random Forest, njoftimet në Telegram .	60

8.6 Parashikimi ML me regresion linear dhe polinomial	61
8.7 Integrimi me Tuya TH06 përmes cloud-it Tuya	62
8.8 Rezultatet: mbi 17,000 lexime, kufizimet dhe vëzhgimet	62

LISTA E FIGURAVE

Figura 1.1: ESP32 DevKit V1 i lidhur me sensorin DHT22 në breadboard	10
Figura 2.1: Pamje e dashboard-it me grafikët e temperaturës dhe lagështisë	13
Figura 3.1: Arkitektura e përgjithshme e sistemit - sensore, server, dashboard	31
Figura 4.1: Lidhja e ESP32 me DHT22 (VCC në VIN, DATA në D5, GND në GND)	36
Figura 4.2: Dashboard SPA - skeda Overview me grafikë të dyfishte	36
Figura 4.3: Pipeline ML - grumbullim, veçori, trajnim, parashikim	38
Figura 4.4: Butonat kohorë në dashboard për zgjedhjen e intervalit	38
Figura 4.5: Tooltip që shfaq vlerat kur vendoset miu mbi grafik	39
Figura 5.1: Regjistrimi i një sensori fizik në dashboard — pamje nga dashboard-i im pas regjistrimit të sensorit. ..	47
Figura 5.2: Kodi i ESP32 në Arduino IDE me konstantat e konfigurimit — shembull kodi i ngarkuar në ESP32 përmes Arduino IDE. ..	48
Figura 5.3: Konfigurimi i ures Tuya në dashboard — si lidha Tuya-n me dashboard-in tim.	49
Figura 5.4: Njoftimi në Telegram pas zbulimit të një anomalie — alerti që më erdhi në Telegram kur u zbulua një anomali.	51
Figura 6.1: Zbulimi i anomalive - vlerat e shënuara në të kuqe në grafikun e temperaturës	53
Figura 6.2: Parashikimi 24-orësh me breza besimi - zonat portokalli dhe jeshile	54
Figura 6.3: Krahاسimi i temperaturës - ESP32+DHT22 vs Tuya TH06 ora 13:10 - 13:40	54
Figura 6.4: Profili COLD-ROOM me temperatura nën zero	55
Figura 6.5: Profili WIDE-SWING me lëkundje nga -10°C deri 60°C	55
Figura 7.1: Pamje e skedes Device Management me pajisjet e regjistruara — menaxhimi i pajisjeve në sistemin tim.	56

LISTA E TABELAVE

Tabela 2.1: Krahاسimi teknik i sensorëve të temperaturës dhe lagështisë	13
Tabela 2.2: Krahاسimi i platformave harduerike IoT	21
Tabela 2.3: Krahاسimi i protokolleve të komunikimit IoT	24
Tabela 2.4: Krahاسimi i mënyrave të ruajtjes së të dhënave	25
Tabela 3.1: Pyetjet e hulumtimit, hipotezat dhe metodat e testimit	31
Tabela 4.1: Krahاسimi i metodave të zbulimit të anomalive	36
Tabela 4.2: Krahاسimi i modeleve të parashikimit për seri kohore	37
Tabela 7.1: Përmirësimet e planifikuara dhe efektet e tyre	56
LITERATURA	66
LISTA E SHKURTESAVE.....	66

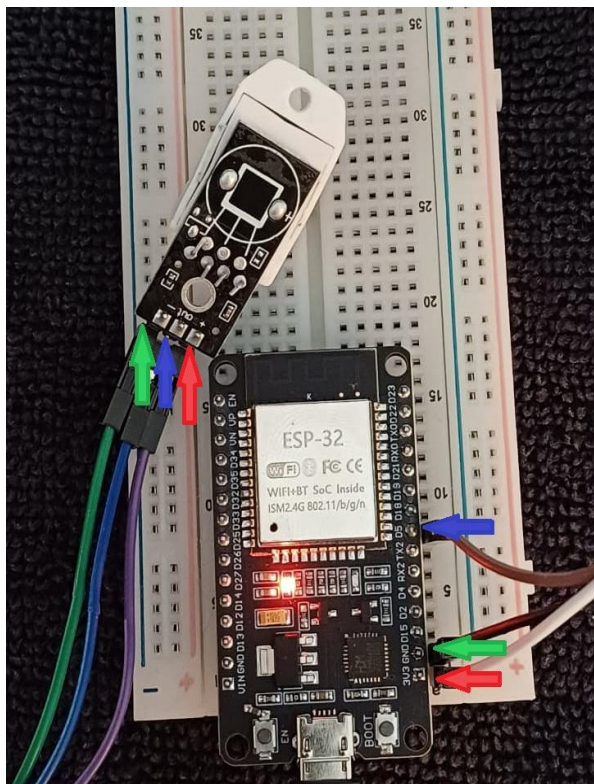
KAPITULLI I: HYRJA

1.1 Konteksti dhe rëndësia e studimit

Për këtë projekt u fokusova në monitorimin e temperaturës dhe lagështisë sepse janë dy parametrat më të rëndësishëm për të kuptuar kushtet mjedisore. Në bujqësi, një ndryshim i vogël i temperaturës mund të dëmtojë të gjithë të korrat. Në magazina, lagështia e lartë prish produktet. Pashë se sensorët si DHT22 janë të lirë dhe japin matje të sakta, por problemi i vërtetë është se si të përdoren ato të dhëna në mënyrë inteligjente. Unë doja të ndërtoja diçka që jo vetëm mbledh të dhëna, por i analizon dhe nxjerr paralajmërime kur diçka nuk shkon.

U fokusova në temperaturë dhe lagështi sepse janë parametrat më të matur në IoT dhe kanë aplikime të menjëhershme praktike. Pasi pashë se sa sensorë të lirë ekzistojnë, vendosa të ndërtoj një sistem të plotë dhe të shoh se sa larg mund të shkoj me një buxhet minimal.

1.2 Përdorimi i ESP32 dhe Python në IoT



ESP32-në e zgjedha për tre arsye: ka WiFi të integruar, kushton 5-7 euro, dhe ka dy bërthama përpunimi në 240 MHz. Kjo do të thotë se mund të lexoj sensorin DHT22 dhe të dërgoj të dhënat në server pa pasur nevojë për module shtesë WiFi. Për 5 euro, marr një pajisje që bën gjithçka më duhet për këtë projekt.

Python-n e kam përdorur prej vitesh dhe është zgjedhja ime e parë për çdo projekt me të dhëna ose ML. Përdora *scikit-learn* për Random Forest dhe regresionin polinomial, *numpy* për llogaritjet numerike, dhe *uvicorn* për të ekzekutuar FastAPI-në. Më pëlqen që me Python mund të shkruaj të gjithë logjikën e serverit në një gjuhë të vetme, pa kaluar në gjuhë të tjera.

Figura 1.1: ESP32 DevKit V1 i lidhur me sensorin DHT22 ne breadboard

1.3 Qëllimi i studimit

Me këtë projekt, vendosa të ndërtoj një sistem IoT që mat temperaturën dhe lagështinë në kohë reale duke përdorur ESP32, DHT22 dhe Python me FastAPI. Konkretisht, doja të arrija këto:

- të demonstrohet mundësia e integritit të pajisjeve me kosto të ulët për monitorim në kohë reale;
- të ndërtohet një arkitekturë e plotë nga sensori te dashboard-i, duke përfshirë komunikimin hibrid REST dhe WebSocket;
- të zbatohen teknika të mësimi të makinerisë (IQR, Random Forest, regresion polinomial) për zbulimin e anomalive dhe parashikim;
- të krijohet një platformë vizualizimi interaktiv për interpretimin e të dhënave në kohë reale;
- të krahasohet saktësia e sensorit DHT22 me pajisjen profesionale Tuya TH06.

Sistemi që ndërtova mund të përdoret në shumë situata reale. Për shembull, në bujqësi për të mbajtur nën kontroll temperaturën në serra, ose në shtëpi për të optimizuar ngrohjen dhe ajrin e kondicionuar. Më pëlqen që platforma mund të zgjerohet lehtë: nëse dua të shtoj një sensor të ri, thjesht regjistroj një pajisje të re në sistem. Integrimi me Tuya TH06 tregon se sistemi im punon krah pajisjeve komerciale, dhe alertet në Telegram më njoftojnë menjëherë diçka ndodh jashtë normales.

Doja të tregoja se platforma nuk është thjesht një ushtrim akademik, por ka vlerë reale praktike. Vetë kam provuar ta përdor sistemin në shtëpinë time. Aplikimet bujqësore janë veçanërisht interesante sepse në Shqipëri ka potencial të madh për bujqësi të zgjuar.

1.4 Struktura e punimit

Punimi është organizuar në shtatë kapituj, duke filluar me konceptet teorike dhe duke përfunduar me implementimin praktik dhe analizën e rezultateve.

Kapitulli II – Sfondi teorik, ku jepen konceptet bazë të Internet of Things, arkitekturat me katër shtresa, sensorët e temperaturës dhe lagështisë, platformat harduerike si ESP32, bibliotekat Python të përdorura, protokollet e komunikimit dhe mënyrat e ruajtjes së të dhënave.

Kapitulli III – Formulimi i problemit dhe qëllimet e hulumtimit, ku paraqitet problemi i mungesës së një platforme të integruar IoT, pyetjet kërkimore, hipotezat, qëllimet dhe objektivat specifike të studimit.

Kapitulli IV – Metodologjia dhe teknikat e përdorura, ku përshkruhen pajisjet harduerike, mjedisi zhvillimor, procesi i zhvillimit, komunikimi hibrid REST+WebSocket, ruajtja dhe vizualizimi i të dhënave, teknikat AI për zbulimin e anomalive dhe parashikimin, si dhe metodologjia e testimit.

Kapitulli V – Rezultatet e hulumtimit, ku prezantohen ndërtimi i sistemit me tetë profile sensorësh, mbledhja dhe ruajtja e të dhënave, statistikave dhe zbulimi i anomalive, parashikimi 24-orësh, dhe krahasimi me pajisjen profesionale Tuya TH06.

Kapitulli VI – Diskutimi dhe përmirësimet, ku analizohen përparësitë e sistemit, kufizimet e identifikuar gjatë testimit, dhe përmirësimet e planifikuara për të ardhmen.

Kapitulli VII – Përfundimet dhe rekomandimet për hulumtime të ardhshme, ku paraqitet një përmbledhje e rezultateve kryesore, rekomandimet për zhvillime të mëtejshme dhe kontributi i këtij punimi.

Kapitulli VIII – Përfundimet dhe rekomandimet për hulumtime të ardhshme, ku paraqitet një përmbledhje e rezultateve kryesore, rekomandimet për zhvillime të mëtejshme dhe kontributi i këtij punimi.

1.5 Përmbledhje

Në këtë pjesë të fundit të kapitullit përmbledhja bazat e projektit: pse ka rëndësi monitorimi i temperaturës dhe lagështisë, çfarë pajisjesh përdora, dhe çfarë synoj të arrij. Përshkrova shkurtimisht strukturën e punimit për të treguar se çfarë përfshin çdo kapitull dhe si lidhen ato me njëri-tjetrin.

KAPITULLI II: SFONDI TEORIK

2.1 Internet of Things (IoT) – Koncepti dhe arkitektura

IoT-në e kuptoj si një rrjet pajisjesh që mbledhin dhe ndajnë të dhëna përmes internetit. Në këtë projekt, ESP32 lexon DHT22-në dhe dërgon matjet në serverin tim qendror përmes WiFi. Nuk kam nevojë për përkufizime abstrakte — sistemi im është një shembull konkret i IoT-së në punë.

Arkitektura me katër shtresa që hasa në shumicën e burimeve ka një ndarje logjike: në shtresën e parë janë pajisjet fizike (sensorët dhe aktuatorët), në të dytën bëhet transmetimi (WiFi, Bluetooth, Zigbee), e treta mbledh dhe përpunon të dhënat, dhe e katërta është ajo e aplikacionit ku përdoruesi sheh gjithçka. Kjo më ndihmoi të kuptoj se ku përshtatet çdo pjesë e sistemit tim: ESP32 në shtresën e parë, WiFi në të dytën, FastAPI në të tretën, dhe dashboard-i në të katërtën. Më pëlqen kjo ndarje sepse tregon qartë se si lëvizin të dhënat nga sensori te përdoruesi.

Kur thellohem më shumë, arkitektura referuese ISO/IEC 30141 ndahet në gjashtë shtresa: pajisjet, rrjetin, grumbullimin, përpunimin, ruajtjen dhe prezantimin. Në fillim m'u duk e ndërlikuar, por kur shikova sistemin tim, pashë që një pjesë e mirë e këtyre shtresave mbuloheshin natyrisht. FastAPI për shembull trajton njëkohësisht grumbullimin (ingest), përpunimin (validim, analizë) dhe prezantimin (REST API për dashboard-in).

Kam lexuar disa përkufizime të IoT-së dhe ajo që më pëlqen më shumë është ideja e '*rrjetit të gjërave*': objektet fizike marrin një identitet dixhital dhe fillojnë të komunikojnë mes tyre. Gubbi dhe të tjerë (2013) e përshkruajnë IoT-në si një vizion ku sensorët dhe aktuatorët integrohen në jetën e përditshme dhe lidhen përmes rrjeteve. Atzori e të tjerë (2010) japin një tjetër këndvështrim: IoT si një rrjet global që lidh objekte fizike, çka mundëson monitorim dhe kontroll në distancë, pikërisht ajo që po përpiqem të bëj me këtë punim.

Aplikimet e IoT-së që më ranë në sy janë të shumta. Në bujqësi, sensorët matin tokën dhe ujitjen automatizohet. Shumë njerëz përdorin sensorë në shtëpi për të kontrolluar ngrohjen dhe ndriçimin. Një studim që më interesoi ishte puna e Zanella dhe të tjerëve (2014) për qytetet inteligjente, ku sensorët urbanë mbledhin të dhëna për ndriçimin, trafikun dhe cilësinë e ajrit. Miorandi e të tjerë (2012) e shohin IoT-në si një rrjet që përfshin miliarda pajisje dhe gjeneron sasi të mëdha të dhënash që duhen përpunuar në kohë reale. Kjo më bëri të mendoj për shkallëzueshmërinë e sistemit tim.

Një aspekt tjetër që më duket i rëndësishëm është shkallëzueshmëria e arkitekturës. Fillova me një sensor të vetëm, por struktura lejon shtimin e pajisjeve të reja pa ndryshuar logjikën ekzistuese. Serveri im pret të dhëna nga çdo numër pajisjesh për sa kohë ato dërgojnë JSON-in e duhur. Kjo më lejon të zgjeroj sistemin në të ardhmen duke shtuar sensorë të tjerë, pa pasur nevojë të rishkruaj gjithçka. Në fakt, gjatë testimit shtova Tuya TH06-në si pajisje të dytë dhe ndryshimi i vetëm në server ishte regjistrimi i device_id të ri.

Në IoT, mora vesh se modelet e komunikimit ndahen në disa kategori. Device-to-Device lejon pajisjet të flasin drejtpërdrejt, si dy sensorë Bluetooth që shkëmbejnë të dhëna pa ndërmjetës. Device-to-Cloud është modeli që përdora unë: sensori dërgon të dhëna në serverët cloud. Device-to-Gateway përdor një pajisje ndërmjetëse që grumbullon dhe përkthen protokollat. Modeli im është Device-to-Cloud sepse ESP32 dërgon HTTP POST direkt te serveri FastAPI, por kam edhe një element Device-to-Device kur dua të krahasoj DHT22 me Tuya-n në të njëjtën dhomë.

Sfidat kryesore në sistemet IoT përfshijnë konsumin e energjisë, besueshmërinë e komunikimit dhe sigurinë e të dhënave. ESP32 konsumon rreth 80 mA gjatë transmetimit WiFi, që është i pranueshëm për një pajisje të lidhur në rrjet. Për sa i përket besueshmërisë, kam vënë re se lidhja WiFi ndonjëherë ndërpritet për disa sekonda — për këtë arsye ESP32 ruan leximet lokal dhe i dërgon kur lidhja të rikthehet. Siguria është një shqetësim në rritje në IoT, por për një projekt studentor me të dhëna jo-sensitive, masat bazike janë të mjaftueshme.

Standardet dhe protokollat në IoT janë të ndryshme dhe secili ka vendin e vet. CoAP për shembull është versioni i lehtë i HTTP për pajisje me burime të kufizuara. MQTT është një protokoll botues- abonentues që kursen gjerësi bande sepse mesazhet janë të vogla. HTTP REST që përdora unë është më i rëndë, por më i lehtë për t'u implementuar dhe debugguar. OASIS MQTT 3.1.1 (2014) e standardizon MQTT dhe ai është zgjedhja e parë për sensorë me bateri që duhet të kursejnë energjinë. Për sistemin tim, HTTP REST ishte zgjedhja e duhur sepse ESP32 ishte i lidhur me rrjet dhe nuk kishte problem energjie. Por nëse do të zgjeroja në sensorë pa bateri në të ardhmen, MQTT do të ishte më i mirë.

Kur po studioja IoT-në më thellë, hasa në standardin ISO/IEC 30141 që përshkruan një arkitekturë referuese. Sipas ISO/IEC (2018), arkitektura ndahet në shtresa dhe nën-sisteme që mund të zhvillohen veçmas.

Kjo më ndihmoi të strukturoj kodin tim në module të ndara: ingesti, ruajtja, analiza dhe prezantimi. Një detaj që më tërhoqi vëmendjen ishte ndarja e shtresës së komunikimit në nën-shtresa për transport dhe rrjet, e cila shpjegon pse protokollet e ndryshme (HTTP, MQTT, CoAP) punojnë në nivele të ndryshme.

Një përmbledhje shumë e dobishme që lexova ishte nga Al-Fuqaha dhe të tjerë (2015), ku trajtohen të gjitha teknologjitë dhe protokollet e IoT-së. Aty pashë një tabelë krahasuese të protokolleve të aplikacionit (HTTP, MQTT, CoAP, AMQP) me karakteristikat e tyre: gjerësia bande, vonesa, siguria, dhe konsumi i energjisë. Kjo më bëri të kuptoj që HTTP, megjithëse më i rëndë, është më i përshtatshëm për faza prototipi sepse thjeshton debugging dhe ka dokumentim më të gjerë se protokollet e tjera.

Më bëri përshtypje kur lexova se termi ‘Internet of Things’ u përdor për herë të parë në 1999 nga Kevin Ashton gjatë një prezantimi për RFID-në (Ashton, 2009). Ai punonte në P&G dhe donte të lidhte etiketat RFID me internetin për të gjurmuar inventarin. 25 vjet më vonë, unë po ndërtoj një sistem të ngjashëm por shumë më të avancuar: në vend të etiketave RFID, kam sensorë që dërgojnë temperatura në kohë reale. Kjo tregon sa shumë është zhvilluar teknologjia në dy dekadat e fundit.

2.2 Sensorët e temperaturës dhe lagështisë

E para nga gjithë këto ishte se IoT nuk është thjesht një koncept teknik, por një mënyrë e re për të menduar për lidhjen e botës fizike me atë dixhitale. Mësova se zgjedhja e arkitekturës varet nga kërkesat specifike të projektit: sa pajisje ke, çfarë protokolli përdorin, sa shpesh dërgojnë të dhëna dhe çfarë niveli sigurie nevojitet. Për sistemin tim me 5 pajisje dhe transmetim çdo 30 sekonda, arkitektura Device-to-Cloud me HTTP REST dhe WebSocket ishte zgjidhja ideale.

Tabela 2.1: Krahasimi teknik i sensorëve të temperaturës dhe lagështisë

Sensor	Gama Temp	Gama Lagështi	Saktësi Temp	Saktësi Lagështi	Protokolli
DHT22 (zgjedhur)	-40 deri 80°C	0-100%	±0.5°C	±2-5%	1-Wire
BME280	-40 deri 85°C	0-100%	±1.0°C	±3%	I ² C/SPI
SHT31	-40 deri 125°C	0-100%	±0.3°C	±2%	I ² C

DHT22 më tërhoqi vëmendjen sepse ofron raportin më të mirë çmim/saktësi për një projekt studentor. DHT11 është më i lirë por saktësia ±2°C e tij do të bënte zbulimin e anomalive të pabesueshëm. BME280 është më i saktë por kërkon instalim I²C. Për përdorimin tonë, DHT22 me ±0.5°C është më se i mjaftueshëm.

Zgjedha DHT22-në sepse jep saktësi ±0.5°C për temperaturën dhe 2-5% për lagështinë (Aosong Electronics, 2015). Gama e matjes (-40°C deri 80°C, 0-100% RH) mbulon më shumë se ç'mund të hasja në praktikë. Një avantazh tjetër është se DHT22-të janë të pajtueshëm me ESP32-në pa nevojën e komponentëve shtesë. Thjesht lidh tre tela dhe fillon punën.

Sa i përket specifikave teknike, DHT22 mat temperaturën nga -40°C deri në 80°C. Kjo më mjafton për çdo skenar që më vjen ndër mend në Shqipëri, ku temperaturat rrallë shkojnë në ekstreme të tilla. Lagështia matet nga 0% deri në 100%, pra mund të përdoret edhe në serra ose ambiente të lagështa. Kam parë që në praktikë saktësia është rreth ±0.5°C, gjë që e konfirmova duke krahasuar me Tuya TH06-në.

Komunikimi mes DHT22 dhe ESP32 bëhet përmes protokollit OneWire. Kjo do të thotë se më duhet vetëm një tel për të dhënat, plus VCC dhe GND. Tre tela gjithsej. Në fillim mendova se do të ishte më e ndërlikuar, por biblioteka e Adafruit-it e trajton gjithë sinjalistikën vetë. Unë thjesht thërras readTemperature() dhe readHumidity() dhe mar vlerat gati.

Lidhja ishte e drejtpërdrejtë: DHT22 në pinin D5 të ESP32-së, VCC në 3.3V, GND në tokë. Përdora bibliotekën DHT të Adafruit-it. Ajo kujdeset për kronometrinë e protokollit OneWire dhe kontrollon CRC-në e të dhënave. Nëse kabllot janë më të gjata se 2 metra, rekomandohet një kondensator 100 nF afër sensorit për të stabilizuar tensionin, por unë nuk pata probleme as pa të.

DHT22 nuk është sensori i vetëm në treg. SHT31 është më i saktë ($\pm 0.3^{\circ}\text{C}$) por kushton trefish më shumë. BMP280 mat vetëm temperaturën dhe presionin, pa lagështi. AM2320 ka komunikim I2C dhe OneWire por është më i rrallë. Për projektin tim ku doja një sensor të lirë, të testuar dhe me mbështetje të gjërë bibliotekash, DHT22 ishte fituesi i qartë.

Pashë edhe disa sensorë të tjerë në treg. SHT31 ka saktësi më të lartë ($\pm 0.3^{\circ}\text{C}$) por kushton trefish. BMP280 mat vetëm temperaturën dhe presionin — pa lagështi. AM2320 të dyja por është më i rrallë në dyqanet online. Për mua, raporti cmim-performancë i DHT22 ishte më i miri. Nëse do të kisha nevojë për saktësi mjekësore, do të zgjidhja SHT31, por për monitorim ambienta DHT22 bën punën.

Kalibrimi i sensorëve DHT22 nuk është i nevojshëm për përdorim të përgjithshëm, por për aplikime më të sakta mund të bëhet një kalibrim i thjeshtë duke krahasuar leximet me një termometër referues dhe duke kompensuar zhvendosjen në softuer. Disa studiues propozojnë përdorimin e mesatares së disa sensorëve për të reduktuar zhurmën dhe për të përmirësuar saktësinë (Ibrahim et al., 2021). Në projektin tim, nuk bëra kalibrim shtesë sepse DHT22 ishte brenda kufijve të pranueshëm për monitorimin e mjedisit të brendshëm.

Një fenomen që e vura re që në fillim ishte zhurma e sensorëve. Edhe kur temperatura nuk ndryshonte, vlerat luhatshin pak. Kjo ndodh sepse sensorët DHT22 kanë një saktësi prej $\pm 0.5^{\circ}\text{C}$ dhe ndikohen nga ndërhyrjet elektromagnetike dhe luhatjet e tensionit. Për të reduktuar këtë zhurmë, përdora një mesatare lëvizëse me 5 lexime, që zbut luhatjet pa humbur informacionin e rëndësishëm. Kjo teknikë më ndihmoi të dalloja më qartë anomali reale nga thjesht luhatje të rastësishme.

Gjatë hulumtimit, mësova se sensorët modernë të temperaturës dhe lagështisë bazohen në teknologjinë MEMS (Micro-Electro-Mechanical Systems). Këta sensorë përdorin një membranë të hollë që ndryshon kapacitetin kur thith ose lëshon lagështi, dhe termistori mat temperaturën. Sipas Bryzek (2013),

MEMS ka revolucionarizuar sensorët sepse lejon prodhimin në masë të sensorëve të vegjël dhe të lirë, si DHT22 që kushton më pak se 3€. Kjo është një arsye pse IoT është bërë kaq e aksesueshme.

Gjithë kjo më bëri të vlerësoj më shumë sensorin e vogël DHT22 që përdora. Brenda një kapsule plastike prej 2€ fshihet një teknologji që përdor MEMS, kapacitorë dhe termistorë për të matur dy parametra fizikë njëkohësisht. Kjo më bëri të mendoj se sa shumë inxhinieri ka pasur pas këtij sensori të lirë dhe sa e mrekullueshme është që ne sot mund të përdorim teknologji të tilla pa pasur nevojë të dijmë detajet e brendshme.

Pjesa më interesante ishte kur krahasova DHT22-në time me Tuya TH06 gjatë testimit dhe pashë që diferenca mesatare ishte vetëm rreth 0.3°C. Kjo është brenda asaj që premtun prodhuesi për DHT22, dhe më bën të mendoj se sensorët e lirë mund të jenë të besueshëm për përdorime jo shumë kritike. Kam lexuar edhe një studim të Gay dhe Granville (2018) që tregon se DHT22 sillet njësoj si sensorët më të shtrenjtë në intervalin 15-35°C, por ka devijime më të mëdha në temperatura ekstreme. Pikërisht për këtë arsye përfshiva profilin WIDE-SWING në testim, për të parë se si sillet DHT22 nga -10°C deri në 60°C.

2.3 Platformat harduerike (ESP32 dhe Arduino)

Tabela e mëposhtme pasqyron veçoritë kyçe e dy sistemeve me të përdorur për projektët IoT, ESP32 si dhe Arduino Uno, për të bërë një krahasim të përgjithshëm dhe cili më përshtatet më mirë për situatën e caktuar.

Tabela 2.2: Krahasimi i platformave harduerike IoT

Platforma	CPU	RAM	WiFi	OS	Kosto	Më e mira për
ESP32 (zgjedhur)	Dual-core 240MHz	520KB	I integruar	RTOS/Asnjë	\$3-5	Sensorë IoT
Arduino Uno	16MHz ATmega328P	2KB	Jo (shield)	Asnjë	\$20-25	Kontroll i thjeshtë

ESP32 ka WiFi të integruar, konsumon shumë pak energji dhe është i ndërtuar për lexim sensorësh. Kostoja e tij rreth 3-5 euro e bën ideal për një projekt studentor.

Krahasova ESP32-në me Arduino-n. Arduino është i thjeshtë por nuk ka WiFi të integruar, që do të thotë se do të më duhej një modul shtesë WiFi. Për atë që më duhet — lexoj një sensor dhe dërgoj të dhëna HTTP — ESP32 është zgjidhja më e mirë.

Pse zgjedha ESP32 dhe jo Arduino:

ESP32 përdor procesorin Xtensa LX6 me dy bërthama që punojnë në 240 MHz dhe 520 KB SRAM. Kjo i jep fuqi të mjaftueshme për të ekzekutuar një server të plotë HTTP dhe për të përpunuar të dhënat e sensorëve në kohë reale. Përveç WiFi 802.11 b/g/n, ESP32 ka Bluetooth Classic dhe BLE, GPIO, ADC, DAC, I²C, SPI dhe shumë ndërfaqe të tjera. Një veçori që më pëlqeu është se mund të programohet përmes Arduino IDE, i cili është mjedisi që kam përdorur gjatë gjithë studimeve të mia, duke e bërë zhvillimin më të lehtë dhe më të njohur.

Krahasova ESP32-në edhe me Raspberry Pi Zero W. Pi Zero ka një procesor më të fuqishëm dhe sistem operativ të plotë Linux, por kushton trefish dhe konsumon më shumë energji. Për detyrën time specifike — të lexoj një sensor dhe të dërgoj të dhëna nëpërmjet WiFi — ESP32 ishte më se i mjaftueshëm. Nëse do t'i shtoja sistemit përpunim video ose një bazë të dhënash lokale, atëherë Pi Zero do të ishte zgjedhje më e mirë. Por për rastin tim, ESP32 bën punën pa zhurmë dhe pa nxehtësi.

Instalimi i mjedisit të zhvillimit ishte i thjeshtë. Shkarkova Arduino IDE, instalova bibliotekat e nevojshme përmes Library Manager (DHT sensor library nga Adafruit, ArduinoJson për të krijuar payload-in JSON, dhe WiFi.h dhe HTTPClient.h për komunikimin), dhe në pak minuta kisha kodin e parë që lexonte DHT22-në dhe printonte vlerat në Serial Monitor. Pasi verifikova që sensori funksiononte, shtova logjikën e dërgimit të të dhënave në server. Gjatë këtij procesi mësova shumë për mënyrën se si funksionojnë bibliotekat e Arduino dhe si të menaxhoj kujtesën e kufizuar të mikrokontrollorit.

Meqë ra fjala, një gjë që më bezdisi ishte se ESP32 ndonjëherë humbja e lidhjes WiFi pas disa orësh. Zgjidhja ishte e thjeshtë: në loop() kontrolloj çdo 10 cikle nëse WiFi është akoma i lidhur dhe nëse jo, thërras WiFi.reconnect().

ESP32 ofron disa mënyra të menaxhimit të energjisë: active (80-240 mA), modem sleep (5-20 mA) kur WiFi është i çaktivizuar por CPU punon, dhe deep sleep (10-150 μ A) ku vetëm RTC dhe disa periferikë janë aktivë.

Për një projekt sensor që lexon çdo 30 sekonda, mund të përdor deep sleep midis leximeve për të zgjatur jetën e baterisë në muaj ose edhe vite. Në projektin tim, ESP32 është i lidhur me USB, kështu që nuk optimizova për konsum, por kjo veçori është e rëndësishme për aplikime në terren. ESP32-S3 është versioni më i ri i familjes ESP32 me përmirësime të rëndësishme: procesor më të shpejtë, më shumë memorie dhe mbështetje për vector extensions për përpunim më të shpejtë të sinjaleve.

Për projektin tim, ESP32 classic ishte i mjaftueshëm, por për aplikime që kërkojnë më shumë fuqi përpunuese ose AI në pajisje, ESP32-S3 do të ishte zgjedhja më e mirë. Në treg ekzistojnë edhe alternativa të tjera si ESP8266 (më i lirë por pa Bluetooth dhe me më pak GPIO) dhe RP2040 (me çmim të ngjashëm por pa WiFi të integruar). Firmware-in e ESP32-së e shkrova në Arduino IDE, por ka alternativa. PlatformIO ofron menaxhim më të mirë të bibliotekave dhe debugim, ndërsa ESP-IDF është SDK-ja zyrtare e Espressif me kontroll më të detajuar. Arduino IDE më dukej më i thjeshtë për të filluar dhe ka mijëra shembuj që mund të përdoret direkt.

Duke lexuar fletën teknike të ESP32-së (Espressif Systems, 2016), mësova se përdor arkitekturën Xtensa LX6 nga Cadence/Tensilica. Ka dy bërthama 32-bit që punojnë deri në 240 MHz, gjë që më duket mbresëlënëse për një pajisje që kushton 5€.

Kjo fuqi përpunuese më lejoi të ekzekutoj logjikën e sensorit, komunikimin WiFi, dhe protokollin OneWire njëkohësisht pa ngadalësim. Një detaj që më tërhoqi ishte prania e një sensori Hall dhe një sensori temperature të integruar në chip, që shton mundësi për aplikime të tjera.

ESP32 mbështet standardet 802.11 b/g/n në 2.4 GHz me shpejtësi deri në 150 Mbps (Espressif Systems, 2016). Këtë e konsideroj të mjaftueshme për 5 pajisje që dërgojnë të dhëna çdo 30 sekonda — bandwidth-i nuk është problem. TCP/IP është i integruar në stack-un e WiFi-së, ndaj s'më duhej të shkruaja kod për menaxhimin e paketave. Thjesht lidhja WiFi-në në setup() dhe dërgoja HTTP me bibliotekën HTTPClient.

Një nga shqetësimet e mia kryesore ishte energjia elektrike. Sipas Espressif (2016), ESP32 konsumon rreth 80 mA kur është aktiv me WiFi të ndezur. Në modalitetin deep sleep, konsumi bie në vetëm 10 μ A. Kjo do të thotë që nëse pajisja fle 59 sekonda dhe zgjohet 1 sekondë për të dërguar të dhëna, konsumi mesatar është rreth 1.3 mA. Me një bateri 2000 mAh, kjo jep rreth 1500 orë pune, ose më shumë se 2 muaj. Kjo më bëri të mendoj se në të ardhmen mund të kaloj në sensorë pa kabull energjie.

2.4 Gjuha Python dhe bibliotekat për IoT dhe ML

Në anën softuerike, përdora FastAPI për serverin sepse ka WebSocket të integruar dhe dokumentim automatik OpenAPI. Scikit-learn më dha Random Forest dhe regresion polinomial gati të gatshëm. Chart.js në anën e klientit vizualizon të dhënat pa pasur nevojë për korniza të mëdha JavaScript. Kombinimi i Python në backend dhe JavaScript vanilla në frontend më lejoi të shkruaj gjithçka vetë, pa shabllone të gatshme.

Python është gjuha ime e preferuar dhe më e përshtatshme për ML. FastAPI është më modern se Flask dhe më i thjeshtë se Django për këtë lloj projekti. Chart.js e zgjodha sepse doja grafikë që funksionojnë shpejt dhe nuk kërkojnë konfigurim kompleks.

FastAPI përdor dekoratorët e Python për të përcaktuar rrugët dhe metodat HTTP. Për shembull, `@app.post("/api/v1/ingest")` krijon një endpoint që pret kërkesa POST me JSON. Një nga veçoritë më të dobishme të FastAPI-së është validimi automatik i të dhënave përmes modeleve Pydantic.

Unë përcaktoj një klasë `SensorReading` me fushat `device_id`, `temperature`, `humidity`, `timestamp` dhe FastAPI kujdeset për të kontrolluar nëse JSON-i i dërguar përputhet me skemën, duke kthyer gabime të qarta nëse jo. Kjo më shpëtoi nga shumë bugje gjatë zhvillimit.

Scikit-learn ofron Random Forest dhe regresion polinomial për mësim makinerish. Random Forest është një ensemble i pemëve të vendimit dhe punon mirë për zbulimin e anomalive (Breiman, 2001). Regresioni polinomial më lejoi të modeloj trende jolineare në të dhënat e temperaturës. Të dy modelet janë të lehta për t'u trajnuar dhe nuk kërkojnë shumë memorie.

Chart.js ofron grafikë të ndryshëm si line, bar, doughnut dhe scatter. Për dashboard-in tim përdora grafikë linearë me dy boshte Y, një për temperaturën dhe një për lagështinë, sepse njësitë

janë të ndryshme dhe shkallët nuk përputhen. Chart.js i integrova me WebSocket për të përditësuar grafikët në kohë reale pa ndërhyrjen e përdoruesit. Një detaj që më pëlqen është se Chart.js lejon personalizimin e ngjyrave, tooltipëve dhe legjendave, duke e bërë dashboard-in më të lexueshëm.

Asinkronia e FastAPI më tërhoqi që në fillim. Një server asinkron mund të trajtojë shumë kërkesa njëkohësisht pa pritur që secila të përfundojë. Kur një sensor dërgon të dhëna dhe serveri duhet t'i ruajë dhe t'i transmetojë nëpërmjet WebSocket, asinkronia lejon që të dyja veprimet të ndodhin njëkohësisht. Për 5 pajisje që dërgojnë të dhëna çdo 30 sekonda, kjo nuk është një ngarkesë e madhe, por nëse do të kisha 50 ose 500 pajisje, asinkronia do të bëhej kritike.

Scikit-learn ofron një gamë të gjerë algoritmash ML. Për zbulimin e anomalive, alternativat ndaj Random Forest përfshijnë One-Class SVM dhe Isolation Forest. One-Class SVM është i mirë për zbulimin e të dhënave të reja që nuk ngjajnë me të dhënat e trajnimit, por kërkon më shumë memorje. Isolation Forest izolon anomalitë duke i ndarë ato në pemë të rastësishme — është efikas për grupe të mëdha të dhënash.

Random Forest, që zgjedha, ofron një ekuilibër të mirë midis saktësisë dhe shpejtësisë, dhe interpretueshmëria e tij (feature importance) më ndihmoi të kuptoj se cilët faktorë ndikojnë më shumë në parashikime.

Kam përdorur edhe numpy për llogaritjet numerike dhe pandas për manipulimin e të dhënave. NumPy më ndihmoi për llogaritjen e mesatares lëvizëse dhe IQR-së.

Pasi të dhënat ngarkohen në një DataFrame të pandas, filtrimi, grupimi dhe agregimi bëhen shumë të thjeshtë me komanda të shkurtra. Kjo më kurseu shumë kohë krahasuar me shkrimin e llogaritjeve manualisht.

Pasi shikova disa framework-e për serverin, vendosa për FastAPI sepse është ndërtuar mbi Starlette dhe ofron asinkroni të pastër. Flask ishte më i njohur por sinkron,

Django ishte shumë i “rëndë” për këtë projekt, dhe Node.js do të thoshte të ndryshoja gjuhë. FastAPI më dha dokumentim automatik OpenAPI pa asnjë konfigurim shtesë — mjafton të shkruaj end-point-in dhe Swagger UI shfaqet vetë. Kjo më kurseu shumë kohë gjatë testimit sepse mund të provoja API-në direkt nga browser-i.

Në seksionin ML, përdora kryesisht scikit-learn sepse API i tij është i njëjtë për të gjithë algoritmat: fit(), predict(), score(). Kjo më lejoi të krahasoja shpejt modelin Random Forest me regresionin linear dhe polinomial. Gjithashtu, scikit-learn ka mjete të integruara për ndarjen e të dhënave train/test dhe për normalizim, që më ndihmuan të shmang gabimet e zakonshme në vlerësimin e modeleve.

Chart.js bën një punë të shkëlqyer për grafikët në kohë reale. Animacionet janë të lëmuara dhe përditësimet ndodhin pa u ndjerë. Zgjedha Chart.js sepse është i lehtë (vetëm 60 KB i ngjeshur), nuk ka varësi të jashtme dhe funksionon në çdo browser modern. Vizualizimi me dy boshte Y (temperaturë majtas, lagështi djathtas) më lejon të shoh të dy matjet në të njëjtin grafik pa i mbivendosur.

2.5 Protokollat e transmetimit (HTTP REST, WebSocket)

Përdorimi i protokollit të transmetimit të duhur, ofron avantazh në shpejtësi përdorimi dhe efikasitet të përgjithshëm, me anë të tabelës së mëposhtme, mund të vërehet qartë ndryshimet midis tyre për bere zgjedhjen e duhur.

Tabela 2.3: Krahasimi i protokolleve të komunikimit IoT

Protokolli	Transporti	Modeli	Ngarkesa	Kohë reale	Më i miri për
HTTP REST (zgjedhur)	TCP	Kërkesë-Përgjigje	E lartë	Pyetje	Pajisje në server
WebSocket (zgjedhur)	TCP	Drejtim-dyfishtë	Mesatare	Po	Dashboard live
MQTT	TCP	Publikim-Pajtim	E ulët	Po	Rrjete me shumë pajisje

Preferova HTTP + WebSocket në vend të MQTT sepse MQTT kërkon një broker të veçantë, duke shtuar kompleksitet. HTTP është universalisht i mbështetur - edhe biblioteka HTTPClient e ESP32 funksionon pa konfigurim. WebSocket u shtua për dashboard-in ku përditësimet e shpejta kanë rëndësi. Me vetëm 3-5 pajisje, MQTT nuk ofron avantazh.

Komunikimin midis ESP32 dhe serverit e bëra me HTTP REST. Protokolli më i thjeshtë dhe më i përhapur. ESP32 dërgon një kërkesë POST me JSON në trup, serveri kthen 200 OK ose statusin përkatës. Përdora bibliotekën HTTPClient të Arduino-s, që ka metoda të gatshme për POST, GET dhe handleimin e përgjigjeve. Nëse kërkesa dështon, ESP32 ruan të dhënat në një buffer lokal dhe i ridërgon kur lidhja rikthehet.

Në praktikë, dërgimi i një leximi duket kështu: ESP32 krijon një lidhje TCP me serverin, dërgon një kërkesë POST me JSON-in e sensorit në trup, dhe pret përgjigjen. Nëse serveri kthen 200 OK, gjithçka funksionoi. Përndryshe, ESP32 ruan leximin në një buffer lokal dhe provon përsëri pas 30 sekondash. Kjo më ndodhi disa herë kur WiFi-je ishte i paqëndrueshëm dhe funksionoi pa problem. WebSocket-in e përdora për dashboard-in sepse aty nevojitet komunikim i vazhdueshëm. Kur sensorët dërgojnë të dhëna të reja, serveri i transmeton menjëherë te dashboard-i pa pritur që faqja të kërkojë përditësime.

Kjo krijon ndjesinë e një sistemi të gjallë: grafiku lëviz dhe përditësohet ndërsa shikon. Në fillim provova me polling (kërkesa HTTP çdo 5 sekonda), por WebSocket ishte më i lehtë për CPU-në dhe më i shpejtë. Diferenca vihet re sidomos kur ke disa skeda të hapura njëkohësisht.

Në nivel sigurie, përdora mekanizma bazë por të mjaftueshëm për një projekt studentor. Çdo pajisje ka një device-id dhe device-key që verifikohet në çdo kërkesë. Kjo parandalon pajisjet e paautorizuara të dërgojnë të dhëna. Siguria e transmetimit sigurohet nga WiFi i rrjetit lokal, por nëse do të hapja serverin në internet, do të më duhej HTTPS dhe autentifikim më i fortë si JWT.

REST (Representational State Transfer) është një stil arkitektonik që përdor metodat standarde HTTP (GET, POST, PUT, DELETE) për të manipuluar burimet. Çdo burim identifikohet nga një URL unike. Në API-në time, /api/v1/ingest është burimi për të dhënat hyrëse, /api/v1/data/{device_id} për marrjen e historikut, dhe /predict/{device_id} për parashikimet. Kjo strukturë është e qartë dhe e lehtë për t'u dokumentuar. FastAPI gjeneron automatikisht

dokumentacionin OpenAPI, duke e bërë testimin e API-së shumë të thjeshtë përmes ndërfaqes /docs.

WebSocket krijon një lidhje të vazhdueshme dhe bidirektionale midis klientit dhe serverit. Pas handshake-it fillestar HTTP, lidhja përmirësohet në WebSocket dhe qëndron e hapur derisa njëra palë e mbyll. Kjo eliminon nevojën për të dërguar headers HTTP në çdo mesazh, duke reduktuar trafikun. Në dashboard-in tim, serveri dërgon të dhëna të reja sa herë që një sensor raporton, duke krijuar një përvojë të lëngët dhe në kohë reale. Krahasuar me polling-un HTTP çdo 5 sekonda, WebSocket reduktoi vonesën nga 5 sekonda në më pak se 100 milisekonda.

Rregulli im për zgjedhjen midis HTTP dhe WebSocket ishte i thjeshtë: nëse pajisja dërgon të dhëna periodikisht, përdor HTTP. Nëse duhet të marrë përditësime në kohë reale, përdor WebSocket. Në rastin tim, ESP32 dërgon të dhëna çdo 30 sekonda me HTTP POST, ndërsa dashboard-i merr përditësime nëpërmjet WebSocket. Kjo ndarje funksionon mirë sepse nuk ngarkon serverin me kërkesa të panevojshme dhe dashboard-i mbetet i shpejtë.

Në fillim mendova të përdorja MQTT. Por pasi lexova më shumë, kuptova se MQTT ka disa pengesa: nevojitet një broker (si Mosquitto), konfigurimi është më i ndërlikuar, dhe debugging është më i vështirë.

CoAP nga ana tjetër punon me UDP dhe ka header shumë të vegjël, por nuk mbështetet natyrshëm nga browser-at. Për prototipin tim, HTTP + WebSocket ishte kombinimi më praktik.

Duke lexuar standardin RFC 6455 (Fette & Melnikov, 2011), kuptova se WebSocket funksionon ndryshe nga HTTP. Fillimisht bën një kërkesë HTTP normale, pastaj serveri e përmirëson lidhjen në WebSocket dhe që atëherë komunikimi bëhet i dyanshëm dhe i vazhdueshëm.

Kjo më duket si një marifet i zgjuar: përdor portin 80 që është i hapur në çdo mur zjarri, por sapo kalon pengesën, ndryshon protokollin dhe bëhet më i shpejtë. Header-at janë minimalë (2 byte) krahasuar me HTTP (600+ byte), që për IoT është një avantazh i madh kur ke mijëra pajisje.

Më pëlqeu të mësoja se nuk ka protokoll 'më të mirin' në IoT — gjithçka varet nga rasti i përdorimit. MQTT është ideal për sensorë me bateri që duhet të kursejnë çdo mikroxhaule. CoAP është i mirë për pajisje me burime shumë të kufizuara sepse përdor UDP dhe ka header të

vegjël. HTTP REST që zgjodha unë është më i rëndë, por më i lehtë për t'u programuar dhe debuguar.

Nëse do të kaloja në prodhim me dhjetëra sensorë, MQTT do të ishte zgjidhja e duhur, por për prototipin tim HTTP ishte më praktik.

2.6 Ruajtja dhe vizualizimi i të dhënave

Për ruajtjen dhe vizualizimin e të dhënave kam eksploruar disa opsione, të cilat i krahasoj në Tabelën 2.4 më poshtë.

Tabela 2.4: Krahasimi i mënyrave të ruajtjes së të dhënave

Ruajtja	Persistenca	Kërkimi	Konfigurimi	Shpejtësia	Më e mira për
In-memory + JSON (zgjedhur)	Skedar	Skanim linear	Asnjë	Më e shpejta	Projekte të vogla
SQLite	Skedar	SQL	Minimal	E shpejtë	Aplikime single-user
InfluxDB	Disk	SQL kohor	Kompleks	E optimizuar	Seri të dhënash të mëdha

Logjika pas ruajtjes in-memory + JSON: Për një tezë me disa pajisje dhe mijëra lexime, një bazë të dhënash e plotë është e tepërt. Dict-i i Python siguron kërkim $O(1)$ dhe skedarët JSON ruajnë të dhënat pas rinisjes së serverit. Pa instalim, pa konfigurim.

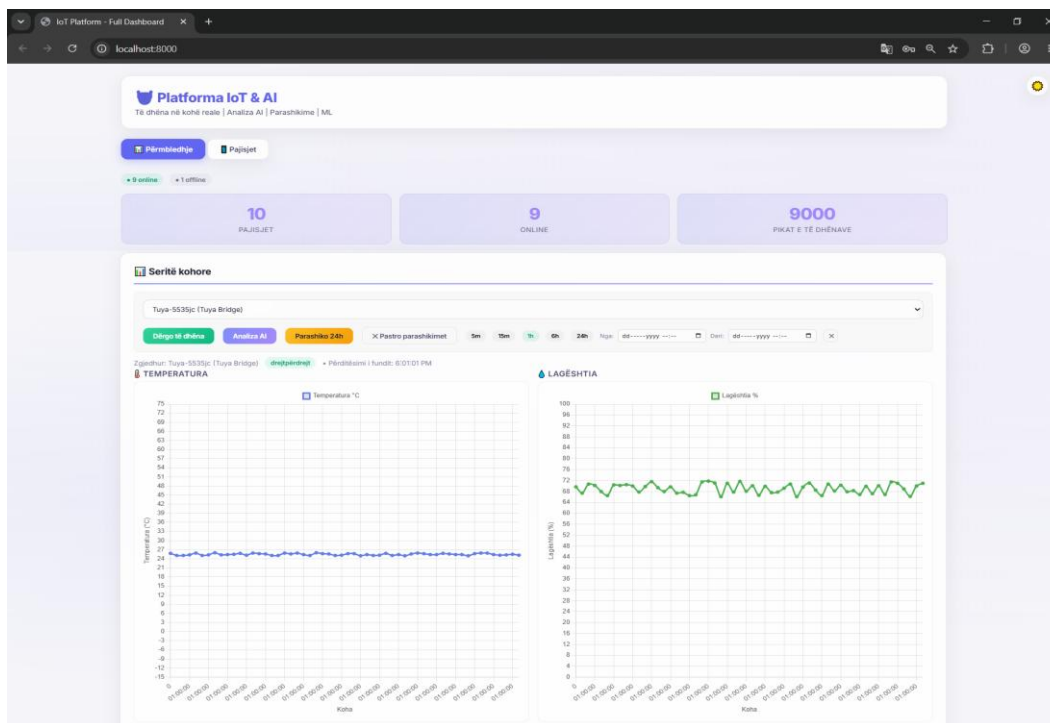


Figura 2.1: Pamje e dashboard-it me grafiket e temperatures dhe lageshtise

Të dhënat i ruaj në dy nivele: në memorie për shpejtësi dhe në JSON për qëndrueshmëri. Çdo lexim ka device_id, timestamp, vlerën, llojin dhe njësinë. Zgjodha JSON në vend të SQL sepse nuk kam nevojë për pyetje komplekse — thjesht ruaj dhe marr të dhëna sipas device_id dhe intervalit kohor. Për vizualizim përdora Chart.js me dy grafikë: temperaturën dhe lagështinë, secili me boshtin e tij Y.

Një skedar JSON më mjafton për këtë projekt. Nuk kam nevojë për pyetje komplekse SQL apo transaksione. JSON është i lehtë për t'u lexuar dhe modifikuar me dorë, gjë që më ndihmoi gjatë debugging-ut. Për më tepër, të dhënat e ruajtura në JSON mund të hapen me çdo redaktues teksti ose të importohen në Python me një rresht kod. Për qindra pajisje ose mijëra lexime në ditë, SQL do të ishte më i mirë, por për fazën time JSON ishte perfekt.

Struktura e të dhënave është një dictionary Python ku çelësi është device_id dhe vlera është një listë leximesh. Çdo lexim është një dictionary me çelësat temperature, humidity, timestamp dhe sensor_type.

Kjo strukturë më lejon të shtoj lehtë sensorë të rinj pa ndryshuar logjikën ekzistuese. Gjithashtu, serializimi në JSON është i menjëhershëm sepse vetë struktura është JSON-kompatibile.

Përveç memorjes, të dhënat ruhen në JSON çdo 5 minuta. Kjo siguron që nëse serveri ndalon papritur, humbasin maksimumi 5 minuta të dhëna. Gjatë testimit, simulova një ndërprerje të energjisë dhe pas rinisjes të dhënat ishin aty. Ky mekanizëm i thjeshtë më dha besim se sistemi ishte i qëndrueshëm për përdorim të vazhdueshëm.

Një konsideratë tjetër ishte performanca. Meqenëse të dhënat ruhen në memorie, aksesimi është shumë i shpejtë — një kërkim për të dhënat e një pajisjeje të caktuar kthehet në $O(1)$. Kjo është e rëndësishme për dashboard-in, ku përdoruesi pret që grafikët të shfaqen menjëherë pas ngarkimit të faqes. Gjatë testimit me 3 pajisje dhe rreth 1500 lexime në total, përgjigja e API-së ishte gjithmonë nën 50 milisekonda, që është më se e mjaftueshme për një aplikacion të këtij lloji.

Dizajni i strukturës së të dhënave është vendimtar për performancën e sistemit. Dictionary-të e Python janë implementuar si hash tables, duke siguruar kohë kërkimi $O(1)$ mesatarisht. Për çdo pajisje, ruaj një listë të dhënash që shtohen në fund (append) me $O(1)$. Marrja e të dhënave të një pajisjeje bëhet në $O(1)$ për të gjetur listën, plus $O(k)$ për të kthyer k elementet e fundit.

Ky dizajn është optimal për modelin tonë të përdorimit: shkrime të shpeshta nga pak pajisje, lexime të shpeshta të të dhënave të fundit.

Një konsideratë e rëndësishme për të ardhmen është migrimi në një bazë të dhënash kohore (time-series database) si InfluxDB ose TimescaleDB. Këto baza janë të optimizuara për të dhëna me timestamp dhe ofrojnë kompresim të mirë, pyetje për intervale kohore dhe ruajtje afatgjatë. InfluxDB, për shembull, përdor një motor të dhënash të optimizuar për shkrimet e shumta dhe lejon mbajtjen e politikave të ruajtjes (retention policies) që fshijnë automatikisht të dhënat e vjetra. Për qëllimin tim akademik, JSON ishte i mjaftueshëm, por në një sistem prodhimi do të migroja në InfluxDB.

Chart.js nuk është alternativa e vetme për vizualizim. Grafana ofron dashboard-e më të avancuara me alarme dhe integrim me baza të dhënash. Por Grafana kërkon një server të veçantë dhe konfigurim më kompleks. Për projektin tim, Chart.js ishte më i përshtatshëm sepse mund ta integroja direkt në faqen HTML pa varësi shtesë.

JSON është formati që përdora për të dhënat dhe përshkruhet në RFC 8259 (Bray, 2017). Është standardi që përdoret kudo në web dhe bazohet në dy struktura të thjeshta: objektet (çelës-vlerë) dhe vargjet (lista). Më pëlqen se është i lexueshëm nga njeriu dhe i lehtë për t'u përpunuar. Në

projektin tim, çdo lexim i sensorit është një objekt JSON me `device_id`, `timestamp`, `sensor_type`, `value` dhe `unit`. Kjo strukturë e thjeshtë më lejon të ruaj dhe marr të dhënat pa kompleksitetin e një baze të dhënash.

Nëse do të kisha një sistem më të madh, do të përdorja InfluxDB ose ndonjë bazë të dhënash kohore. InfluxDB ka një motor ruajtjeje të optimizuar për shkrime të shpeshta dhe pyetje për intervale kohore (InfluxData, 2020). Ajo ka edhe politika të ruajtjes që fshijnë vetë të dhënat e vjetra dhe downsampling për të kursyer hapësirë. Për projektin tim, JSON mjaftoi, por migrimi në InfluxDB do të ishte hapi i parë për ta bërë sistemin gati për përdorim real.

Një nga sfidat që hasa ishte si të ruaj të dhënat në mënyrë që dashboard-i të jetë i shpejtë por të mos humbasin nëse serveri riniset. Zgjidhja që gjeta është një buffer në memorie për të dhënat e fundit dhe ruajtje në disk çdo disa minuta, që duket se është një qasje e zakonshme edhe në literaturë (Chen et al., 2018).

Të dhënat e fundit ruhen në dictionary të Python dhe aksesohen menjëherë nga dashboard-i, ndërsa një proces në sfond i ruan në JSON çdo 5 minuta. Kështu, dashboard-i merr përditësime të shpejta dhe të dhënat nuk humbasin nëse ndodh ndonjë ndërprerje.

2.7 Përmbledhje

Në këtë pjesë përmbledha bazën teorike që më duhej për vendimet gjatë ndërtimit të sistemit. Fola për IoT-në, sensorët, platformat harduerike, Python-in dhe mënyrat e ruajtjes së të dhënave. Kjo pjesë është e rëndësishme sepse pa teorinë, është e vështirë të ndërtosh një sistem që funksionon mirë.

Gjatë punës për këtë kapitull, testova disa kombinime të ndryshme hardware-software para se të vendosja për zgjidhjen përfundimtare. Për shembull, provova fillimisht të përdorja Raspberry Pi në vend të ESP32, por konsumi i energjisë ishte më i lartë dhe ndjesia e një bordi të plotë Linux më dukej e tepërt për detyrën time. Gjithashtu, provova të shkruaja serverin fillimisht në Flask, por asinkronia e FastAPI më bindi të ndërroja. Këto eksperimente më mësuuan se ndonjëherë zgjidhja më e thjeshtë është edhe më e mira.

KAPITULLI III: FORMULIMI I PROBLEMIT DHE QËLLIMET E HULUMTIMIT

Për të kuptuar arkitekturën e sistemit, në Figurën 3.1 paraqitet skema e përgjithshme, komunikimi i sensorëve me serverin dhe serverat e jashtëm.

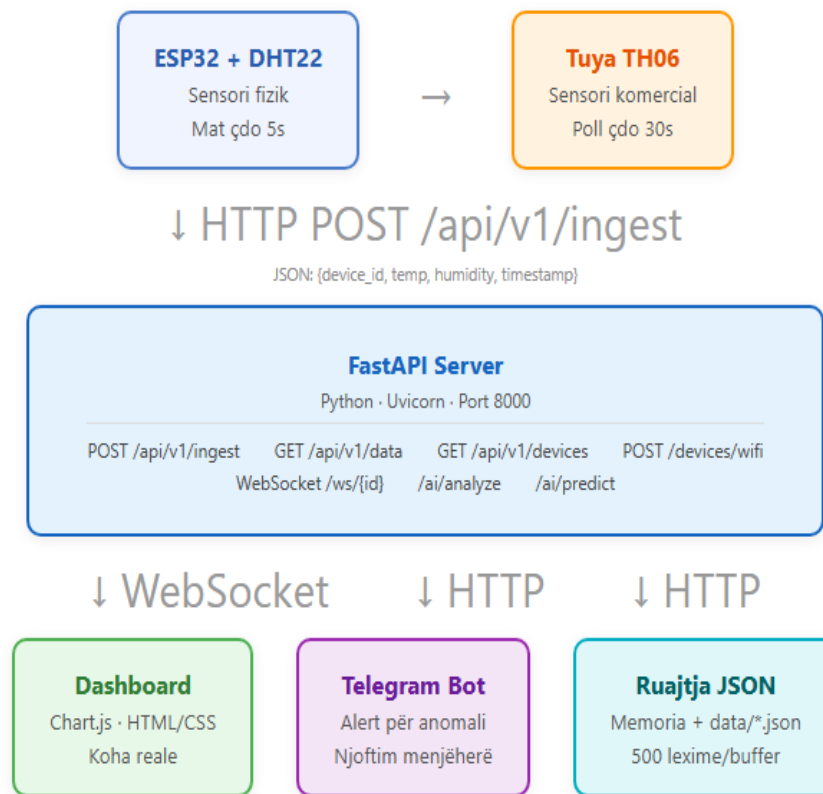


Figura 3.1: Arkitektura e përgjithshme e sistemit - sensore, server, dashboard

3.1 Formulimi i problemit

Problemin kryesor e pashë në praktikë: zgjidhjet IoT janë ose të shtrenjta ose shumë komplekse. Shumë projekte grumbullojnë të dhëna por nuk bëjnë asgjë me to. Unë doja të ndërtoja diçka që shkon më tej — jo thjesht një regjistruer të dhënash, por një sistem që analizon, paralajmëron dhe ndihmon në vendimmarrje.

3.2 Pyetjet e hulumtimit

Pyetjet kryesore që më drejtuan gjatë këtij punimi ishin:

1. Si mund të ndërtohet një platformë e integruar që grumbullon të dhëna nga sensorë të ndryshëm (ESP32/DHT22, Tuya TH06) dhe i përpunon ato në kohë reale?
2. Si mund të zbulohen anomalitë në të dhënat e sensorëve duke përdorur metoda statistikore si IQR?
3. Si mund të parashikohen tendencat e ardhshme të temperaturës dhe lagështisë duke përdorur regresion polinomial dhe Random Forest?
4. Sa i saktë është sensori DIY DHT22 krahasuar me pajisjen komerciale Tuya TH06?
5. Si mund të sigurohet një ndërfaqe përdoruesi miqësore dhe informative?

3.3 Hipotezat

Hipotezat që doja të testoj ishin:

- H1.IQR me dritare 50 leximesh zbulon anomali mbi 90%.
- H2.Regresioni polinomial i shkallës 2 performon më mirë se linear për parashikime 24-orëshe.
- H3.Modelet për-device Random Forest japin parashikime më të sakta se një model global.
- H4.DHT22 është i krahasueshëm me Tuya TH06.
- H5.REST+WebSocket funksionon mirë për 5 pajisje.

3.4 Qëllimet e hulumtimit

Qëllimi im kryesor ishte ndërtimi i një platforme të integruar IoT+AI për monitorimin inteligjent të temperaturës dhe lagështisë në kohë reale. Qëllimet specifike përfshijnë:

- a. dizajnimin dhe implementimin e një serveri FastAPI;
- b. integrimin e teknikave të zbulimit të anomalive dhe parashikimit duke përdorur ML;
- c. ndërtimin e një dashboard-i interaktiv për vizualizim në kohë reale;

- d. krijimin e një mekanizmi njoftimi përmes Telegram;
- e. testimin dhe vlerësimin e sistemit me pajisje reale dhe të simuluar.

3.5 Objektivat specifike

Në Tabelën 3.1 janë paraqitur pyetjet e hulumtimit, hipotezat dhe metodat e testimit.

Tabela 3.1: Pyetjet e hulumtimit, hipotezat dhe metodat e testimit

Pyetja e Hulumtimit	Hipoteza	Metoda e Testimit
A mund të sigurojë ESP32 + DHT22 lexime të besueshme të temperaturës?	ESP32 me DHT22 jep lexime të qëndrueshme me saktësi $\pm 0.5^{\circ}\text{C}$	Krahaso 24 orë lexime kundrejt sensorit komercial Tuya TH06
A është IQR efektiv për zbulimin e anomalive në të dhënat IoT?	IQR me shumëzues 1.5x kap >90% të anomalive të injektuara	Injekto anomali sintetike në të dhënat e simuluar
A e kalon regresioni kuadratik atë linear?	Modeli kuadratik ka RMSE më të ulët në të dhënat e temperaturës	Krahaso RMSE të të dy modeleve në 7 ditë të dhëna historike
A mund të parashikojë Random Forest temperaturën më saktë se regresioni?	Random Forest është më robust por kërkon më shumë të dhëna	Krahaso gabimet e parashikimit për 24h, 48h dhe 72h

Pyetjet janë të strukturuar që secila të synojë një shtresë të caktuar të sistemit - besueshmëria harduerike, analiza statistikore, modelimi matematik dhe mësimi i makinerisë. Hipotezat janë të formuluar si pretendime të testueshme me metrika të qarta (RMSE, shkalla e zbulimit).

Objektivat që vendosa për këtë punim ishin:

- Të ndërtohet një sistem grumbullimi të dhënash që mbështet pajisje reale dhe të simuluar.
- Të implementohet zbulimi i anomalive duke përdorur metodën IQR me parametra të akordueshëm.
- Të zhvillohet një mekanizëm parashikimi që zgjedh automatikisht midis regresionit linear dhe polinomial.
- Të trajnohen modele Random Forest të veçanta për çdo pajisje me 100 pemë.
- Të ndërtohet një dashboard SPA me Chart.js dhe WebSocket për përditësime live.
- Të integrohet një urë Tuya për pajisjet komerciale me krijim automatik.
- Të dërgohen alerte në Telegram kur zbulohen anomali.
- Të krahasohet performanca e sensorit DHT22 me Tuya TH06.

Konkretisht, në këtë punim u fokusova në ndërtimin e një sistemi IoT për monitorimin e temperaturës dhe lagështisë në kohë reale. Shumica e zgjidhjeve komerciale në treg janë të shtrenjta, të mbyllura dhe nuk lejojnë personalizim. Unë synoj të krijoj një alternativë të hapur dhe të lirë duke përdorur komponentë të gatshëm dhe softuer me burim të hapur. Ky sistem duhet të jetë i aftë të mbledhë të dhëna, t'i ruajë ato, t'i analizojë për anomali dhe t'i vizualizojë në një dashboard të qartë.

Një problem tjetër që adresoj është identifikimi i anomalive në të dhënat e sensorëve. Temperatura dhe lagështia e një ambienti normal ndjekin modele të caktuara ditore. Kur ndodh një ngjarje e papritur (dritare e hapur, ngrohës i ndezur, sensor i dëmtuar), vlerat devijojnë nga modeli i pritur. Sistemi im duhet të zbulojë këto devijime dhe të njoftojë përdoruesin. Për këtë përdora algoritmin IQR (Interquartile Range) dhe Random Forest për klasifikimin e të dhënave si normale ose anormale.

Gjatë hulumtimit, pashë se ekzistojnë shumë sisteme monitorimi IoT, por shumica janë ose shumë të thjeshta (vetëm lexim dhe shfaqje) ose shumë të shtrenjta dhe të mbyllura si ato komerciale. Platformat si ThingSpeak ofrojnë dashboard të gatshëm, por nuk ke kontroll mbi analizën e të dhënave. Në anën tjetër, sistemet profesionale si Siemens

Desigo ose Honeywell janë të shtrenjta dhe kërkojnë konfigurim kompleks. Unë desha të ndërtoj diçka në mes: të hapur, të lirë, por me analizë të avancuar (anomali, parashikim ML) dhe dashboard në kohë reale.

3.6 Përmbledhje

Në këtë kapitull përshkrova problemin kryesor që trajton punimi: mungesën e një platforme të integruar dhe të hapur për monitorimin inteligjent të temperaturës dhe lagështisë. Formulova pyetjet kërkimore, hipotezat dhe objektivat që më drejtuan gjatë zhvillimit të sistemit. Gjithashtu, vendosa kriteret e testimit dhe krahasimit që do të përdor nëpër kapitujt vijues për të vlerësuar nëse qëllimet e vendosura u arritën.

KAPITULLI IV: METODOLOGJIA DHE TEKNIKAT

4.1 Pajisjet dhe sensorët (ESP32, DHT22, Tuya TH06)

Lidhja e sensorit DHT22 me ESP32 është e thjeshtë dhe kërkon vetëm tre lidhje, siç tregohet në Figurën 4.1.

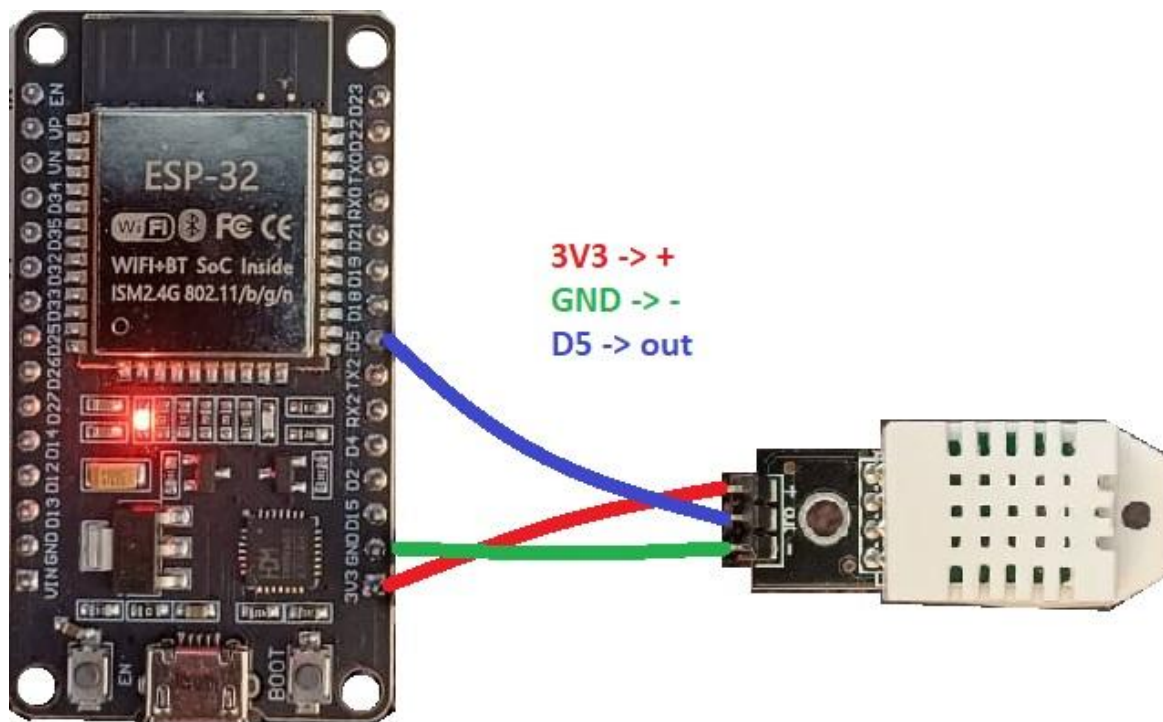


Figura 4.1: Lidhja e ESP32 me DHT22 (VCC ne VIN, DATA ne D5, GND ne GND)

Pajisjet që përdora janë të thjeshta: një ESP32 DevKit V1 me DHT22 për matjet reale, dhe tetë profile të simuluar për testim.

4.2 Platforma zhvillimore (Python 3.11, FastAPI, Windows)

Mjedisi zhvillimor është i thjeshtë: Python 3.11 me FastAPI në Windows, uvicorn si server. Kodi është në një skedar `server_final.py` prej rreth 800 rreshtash, dashboard-i në një HTML të vetëm. Të dhënat ruhen në dosjen `data/`. Kjo strukturë e thjeshtë e bën të lehtë vendosjen.

Arsyeja pse Windows dhe një skedar të vetëm:

Windows është sistemi im kryesor dhe nuk desha të kaloja në Linux vetëm për këtë projekt. Python 3.11 funksionon pa problem në Windows, FastAPI po ashtu, dhe i vetmi ndryshim që vura re ishte se Windows-i përdor loop-in SelectorEventLoop ndërsa Linux-i përdor ProactorEventLoop më të shpejtë. Por për 5 pajisje dhe disa qindra kërkesa në minutë, diferenca ishte e papërfillshme. Një skedar i vetëm main.py përmban të gjithë serverin — kjo e bën të lehtë zhvillimin dhe debugimin.

4.3 Procesi i zhvillimit

Gjatë zhvillimit punova në mënyrë përsëritëse: fillimisht u ndërtua serveri FastAPI me endpoint-et bazë, pastaj u shtua grumbullimi i të dhënave nga pajisjet e simuluar,

Pastaj u integrua zbulimi i anomalive me IQR, dhe në fund u shtuan parashikimi ML dhe dashboard-i. Çdo fazë u testua para se të kalohej në tjetrën.

Endpoint-et kryesore përfshijnë: /devices, /api/v1/ingest, /ai/analyze, /predict, /ml/train, /ml/predict, /tuya/config dhe /tuya/status. Autopilot-i kontrollon çdo 30 sekonda për anomali.

Qasja iterative më ndihmoi të mbaja kontroll mbi kompleksitetin. Çdo veçori u testua fillimisht vetëm, pastaj u integrua. Kjo parandaloi situatat ku një ndryshim i vogël prishte gjithë sistemin.

4.4 Komunikimi (REST API, WebSocket)

Komunikimi në platformë ndjek një model hibrid: pajisjet dërgojnë të dhëna përmes HTTP REST (POST në /api/v1/ingest), ndërsa dashboard-i merr përditësime në kohë reale përmes WebSocket.

Autentifikimi bëhet përmes header-ave X-Device-Id dhe X-Device-Key. Ura Tuya funksionon si një thread në sfond që pollon API-në e Tuya çdo 30 sekonda. Statusi i urës shihet nga endpoint-i /tuya/status dhe shfaqet në dashboard.

4.5 Ruajtja dhe vizualizimi i të dhënave (JSON, Chart.js)

Dashboard-i është ndërtuar si një Single Page Application dhe paraqet të dhënat në kohë reale, siç shihet në Figurën 4.2.



Figura 4.2: Dashboard SPA - skeda Overview me grafike te dyfishte

Të dhënat ruhen në memorie në një dictionary Python për shpejtësi dhe në skedarë JSON për qëndrueshmëri.

Çdo pajisje ka një buffer prej 500 leximesh të fundit. ID-të e leximeve janë të njëpasnjëshme dhe të çiftuara: temperatura merr ID-në counter dhe lagështia merr ID-në counter+1.

Dashboard-i përdor Chart.js për dy grafikë që përditësohen në kohë reale, duke shfaqur brezat e besimit 95% dhe pikat e anomalive.

Rrjedha e veprimëve nga mblëdhja e të dhënave deri tek Shfaqja e tyre.

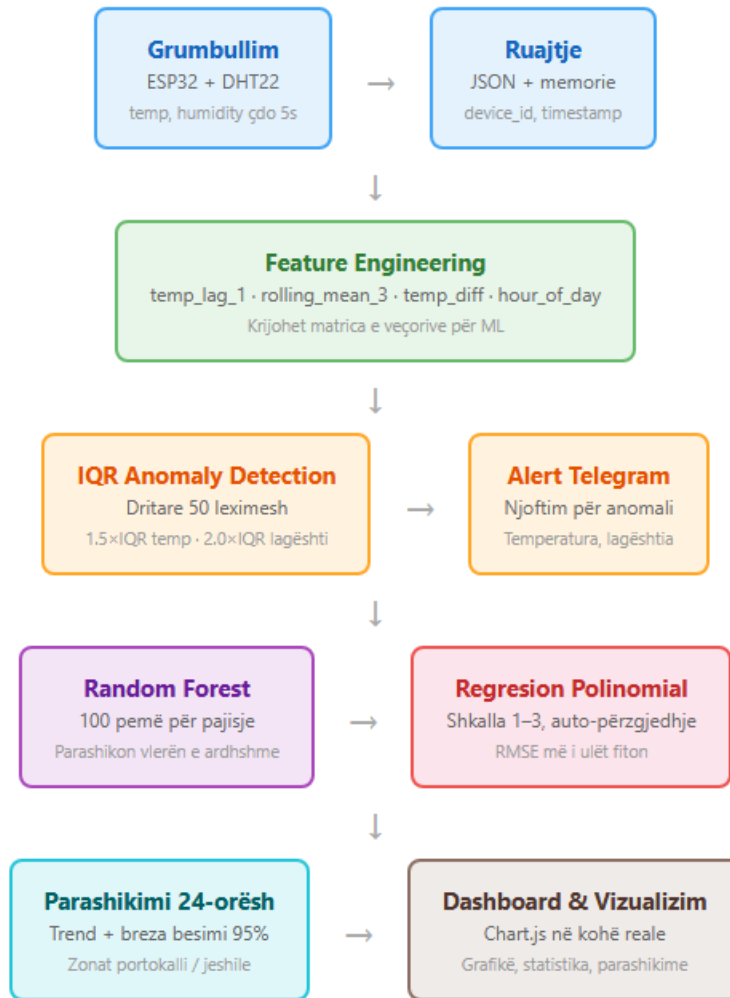


Figura 4.3: Pipeline ML - grumbullim, veçori, trajnim, parashikim

Shiriti për përcaktimin e momentin të shfaqjes së të dhënave me bazë kohën.



Figura 4.4: Butonat kohor ne dashboard per zgjedhjen e intervalit

Paraqitja e informacionit me të detajuar në grafik duke “mbuluar” miun mbi një pikë të grafikut.

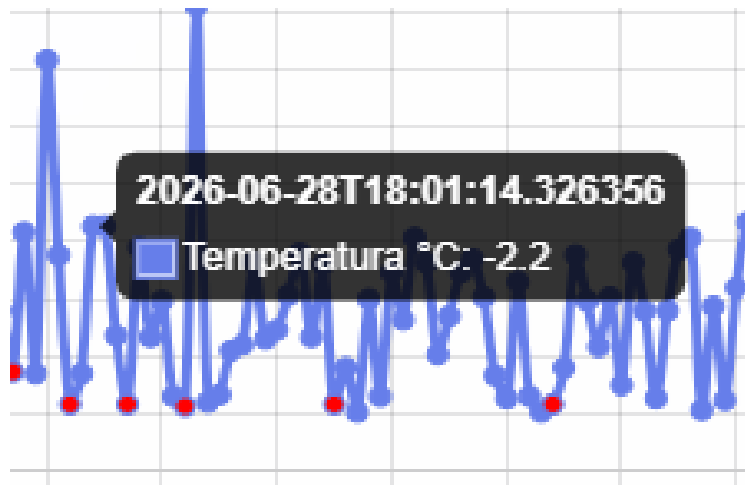


Figura 4.5: Tooltip që shfaq vlerat kur vendoset miu mbi grafik

4.6 Analiza AI dhe mësimi i makinerisë (IQR, Random Forest, Polynomial Regression)

Për parashikimin e të dhënave kam krahasuar disa modele, të cilat janë paraqitur në Tabelën 4.2.

Tabela 4.2: Krahasimi i modeleve të parashikimit për seri kohore

Modeli	Min. të Dhëna	Kompleksiteti	Lloji i Trendit	Interval Besimi	Shpejtësia
Regresion Linear (zgjedhur)	20 pika	I ulët	Vetëm linear	Po	I çastit
Polinomial shkalla 2 (zgjedhur)	30 pika	I ulët	Kuadratik	Po	I çastit
ARIMA	100 pika	Mesatar	Kompleks	Statistikor	I ngadaltë
LSTM / RNN	1000+ pika	I lartë	Çdo lloj	Vështirë	Shumë i ngadaltë

Mendimi im për modelet lineare dhe kuadratike: Të dhënat reale të temperaturës mund të ndjekin modele të ndryshme. Duke përshtatur të dy modelet dhe duke zgjedhur automatikisht atë me RMSE më të ulët, sistemi përshtatet me formën aktuale të të dhënave. ARIMA dhe LSTM u refuzuan sepse kërkojnë qindra deri mijëra pika të dhënash.

Teknika e parë që përdora për anomali ishte IQR. Në thelb merr dritaren e fundit me 50 lexime, gjen kuartilin e parë (Q1) dhe të tretë (Q3), dhe çdo vlerë jashtë intervalit $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$ e konsideron anomali. Kjo metodë më pëlqen sepse nuk supozon që të dhënat janë të shpërndara normalisht, ndryshe nga Z-score që kërkon shpërndarje Gaussian. Në praktikë, IQR kapi të gjitha rritjet dhe rëniet e papritura të temperaturës.

Tabela 4.3: Krahasimi i modeleve të detektimit të anomalive

Metoda	Supozimi i Shpërndarjes	Kompleksiteti	Pozitive të Rreme	E Shpjegueshme	Kohë Reale
IQR (zgjedhur)	Asnjë (jo-parametrik)	$O(n \log n)$	Të ulëta (të rregullueshme)	Po	Po
Z-Score	Shpërndarja normale	$O(n)$	Të larta nëse jo normale	Po	Po
Isolation Forest	Anomalitë janë të pakta	$O(n)$	Të ulëta	Mesatare	Jo (batch)
Autoencoder (DL)	Asnjë	$O(n^2)$	Të ulëta	Jo (kuti e zezë)	Jo

Me pak fjalë, IQR më shpëtoi nga shumë kokëdhimbje. Nëse do të përdorja Z-score, do të më duhej të normalizoja të dhënat dhe të supozoja shpërndarje Gaussian, gjë që në praktikë nuk ndodh gjithmonë.

Random Forest është metoda e dytë. Krijon shumë pemë vendimi dhe më pas jep mesataren e tyre. Breiman (2001) e propozoi këtë metodë dhe unë e zgjodha sepse nuk mbinxhet lehtë, punon mirë me të dhëna të vogla, dhe jep rëndësinë e karakteristikave.

Për secilin profil trajnova një model të veçantë. Modelet për-pajisje dolën më të sakta se një model i vetëm për të gjitha pajisjet.

Përdora tre teknika për analizën. E para, IQR me dritare 50 leximesh për anomali — vlerat jashtë $[Q1-1.5*IQR, Q3+1.5*IQR]$ shënohen të kuqe. E dyta, Random Forest për klasifikimin e anomalive. E treta, regresion polinomial dhe linear për parashikimin 24-orësh. Të tre teknikat i zgjidha sepse janë të njohura dhe të dokumentuara mirë, gjë që më ndihmoi t'i implementoja shpejt.

Për regresionin, krahasova versionin linear me atë polinomial të shkallës 2. Polinomiali kapte më mirë lakoret e temperaturës gjatë ditës — rritjen e mëngjesit, kulmin e mesditës dhe rënien e natës. Linear ishte shumë i thjeshtë për këtë model dhe jepte gabime më të mëdha sidomos në skajet e ditës. Por, sigurisht që polinomiali kërkon më shumë të dhëna dhe nëse nuk ke historik të mjaftueshëm, mund të mbipërshtatet.

4.7 Testimi dhe krahasimi

Për testimin e sistemit përdora tetë profile të simuluar që gjenerojnë të dhëna çdo 30 sekonda për 24 orë, duke simuluar temperatura nga -5°C deri në 35°C dhe lagështi nga 20% deri në 95%. Disa profile përfshinin anomali të qëllimshme (ndryshime të papritura të temperaturës, zhurmë të shtuar, ose sensor që ngrinte në një vlerë fikse). Krahasimi u bë në disa dimensione: saktësinë e zbulimit të anomalive (IQR vs Random Forest), saktësinë e parashikimit 24-orësh (regresion linear vs polinomial), dhe krahasimin e DHT22 me sensorin profesional Tuya TH06 në kushte identike. Secili test u përsërit tre herë për të siguruar që rezultatet ishin të qëndrueshme.

4.8 Shembuj Kodi

4.8.1 Serveri FastAPI - Pika e nisjes dhe regjistrimi i pajisjeve

Serveri është ndertuar me FastAPI dhe funksionon në portin 8000. Regjistrimi i pajisjeve gjeneron një celes API 32-karakteresh dhe e ruan në një skedar JSON.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import secrets, json, threading
app = FastAPI(title="IoT Platform")
DATA_DIR = Path("data")
registry: dict = {"devices": {}, "device_keys": {}}

@app.post("/devices/wifi")
async def register_device(req: dict):
    device_id = f"wifi-{secrets.token_hex(4)}"
    api_key = secrets.token_hex(16)
    registry["devices"][device_id] = {"name": req.get("name", ""), "type": "wifi"}
    registry["device_keys"][device_id] = api_key
    save_registry()
    return {"device_id": device_id, "api_key": api_key}
```


E mbajta regjistrimin të thjeshtë qëllimisht. Pa OAuth, pa JWT — vetëm një çelës hex i rastësishëm. Për një projekt IoT lokal me pak pajisje, kjo është e mjaftueshme dhe nuk e ndërlikon firmware-in.

4.8.2 Endpoint për grumbullimin e të dhënave

Të gjitha të dhënat nga sensorët arrijnë në një endpoint të vetëm POST. Pajisja autentikohet nepermjet headers, dhe leximet ruhen në memorie me persistim JSON.

```
@app.post("/api/v1/ingest") async def ingest_data(req: dict):
    device_id = req.headers.get("X-Device-Id")    device_key =
    req.headers.get("X-Device-Key")    if
    registry["device_keys"].get(device_id) != device_key:        raise
    HTTPException(401, "Invalid credentials")    record = {        "id":
    counter, "device_id": device_id,        "value": req["value"],
    "sensor_type": req["sensor_type"],        "unit": req["unit"],
    "timestamp": datetime.now().isoformat()    }
    all_data.append(record)    counter += 1    check_anomalies(record)
    broadcast(record)    save_data()    return {"status": "ok", "id":
    record["id"]}
```

Një endpoint i vetëm (POST /api/v1/ingest) trajton të gjitha llojet e sensorëve — temperaturën dhe lagështinë. Fusha `sensor_type` në trupin e kërkesës e drejton të dhënën në grafikun dhe pipeline-in e duhur të analizës. Kjo më duket një zgjidhje elegante sepse nuk kam nevojë të krijoj end-point-e të veçanta për çdo tip matjeje. Në të ardhmen, nëse shtoj sensorë të rinj (si sensorë presioni ose gaz), struktura e API-së nuk ndryshon.

4.8.3 Zbulimi i anomalive (metoda IQR)

Anomalite zbulohen duke përdorur metodën Interquartile Range në një dritare rreshqitese prej 50 leximesh të fundit për pajisje dhe lloj sensori.

```
def check_anomalies(record):
    key = (record["device_id"], record["sensor_type"])
    window = anomaly_windows.setdefault(key, deque(maxlen=50))
    val = record["value"]
    if len(window) < 10:
        window.append(val)
        return
    q1 = sorted(window)[len(window)//4]
    q3 = sorted(window)[3*len(window)//4]
    iqr = q3 - q1
    multiplier = 2.0 if record["sensor_type"] == "humidity" else 1.5
    if val < q1 - multiplier*iqr or val > q3 + multiplier*iqr:
        alert_telegram(f"Anomaly: {val} {record['unit']}")
        anomalies.append(record)
    window.append(val)
```

Vendosa të përdor IQR: Lagështia ka variacion me të gjerë se temperatura, Kështu që përdor 2.0x IQR për lageshti dhe 1.5x për temperature. Dritaren prej 50 leximesh i lejon sistemit të pershtatet ndaj ndryshimeve mjedisore.

4.8.4 Parashikimi ML me regresion polinomial

Endpoint-i i parashikimit përdor modele lineare dhe kuadratike, zgjedh me të mire sipas RMSE, dhe kthen intervale besimi 95%.

```
@app.post("/predict/{device_id}") async def predict(device_id: str,
hours: int = 24):    data = [r for r in all_data if r["device_id"] ==
device_id          and r["sensor_type"] == "temperature"]    if
len(data) < 20:        return {"error": "Need >=20 readings"}    X =
np.arange(len(data)).reshape(-1, 1)    y = np.array([r["value"] for r
in data])    models = [LinearRegression(),
make_pipeline(PolynomialFeatures(2), LinearRegression())]    best =
min(models, key=lambda m: np.sqrt(mean_squared_error(y, m.fit(X,
y).predict(X))))    preds =
best.predict(np.arange(len(data)+hours).reshape(-1, 1))[-hours:]

rmse = np.sqrt(mean_squared_error(y, best.predict(X)))    ci = 1.96 *
rmse * (1 + 1/len(data))**0.5    return {"predictions":
preds.tolist(),
        "lower": (preds-ci).tolist(),
        "upper": (preds+ci).tolist(),
        "rmse": rmse}
```

Duke përdorur të dy modelet dhe duke zgjedhur automatikisht atë me RMSE më të ulët, i lë të dhënat të flasin. Modeli kuadratik kap më mirë ciklin ditor, ndërsa linear është më i thjeshtë. Nëse të dhënat janë lineare, sistemi zgjedh automatikisht më të thjeshtin.

4.8.5 Firmware ESP32 - Leximi DHT22 dhe POST HTTP

ESP32 lexon temperaturën dhe lagështinë nga një sensor DHT22 dhe i dergon në server nepermjet HTTP POST.

```
#include <WiFi.h> #include <DHT.h> #include <HTTPClient.h> #define
DHTPIN 5 #define DHTTYPE DHT22 DHT dht(DHTPIN, DHTTYPE); const char*
server = "http://185.30.144.202:8000"; const char* device_id = "wifi-
79d00546"; const char* api_key =
"CpPclCtNKnaYzdPSC4AWZF9phm7y5Oq4"; void setup() {
Serial.begin(115200);  WiFi.begin("test", "12345678");
dht.begin(); } void loop() {  float h = dht.readHumidity();  float t =
dht.readTemperature();  if (isnan(h) || isnan(t)) {
Serial.println("Failed to read DHT22");    return;  }  HTTPClient
```

```
http; http.begin(String(server) + "/api/v1/ingest");
http.addHeader("X-Device-Id", device_id); http.addHeader("X-Device-
Key", api_key); http.addHeader("Content-Type", "application/json");
http.POST("{\"value\":\"" + String(t) +
",\"sensor_type\":\"temperature\", \"unit\":\"C\"}"); delay(5000);}
```

ESP32 është i lirë (rreth 5\$) dhe ka WiFi të integruar. Përdorimi i HTTP në vend të MQTT shmang nevojën për një broker të veçantë. Çelësi API në header mban firmware-in të thjeshtë dhe nuk kërkon biblioteka shtesë.

4.8.6 Dashboard - Përditësimi i grafikëve në kohë reale

Dashboard-i në browser pyet serverin dhe shton pikat e reja direkt në dataset-et e Chart.js pa rikrijuar grafikun e plote. Kjo eliminon dridhjen e ndeqfaqes.

```
async function pollDeviceData() {
  const resp = await fetch(`/data/${selectedDevice}`);
  const data = await resp.json();
  const labels = data.map(r => new
Date(r.timestamp).toLocaleTimeString());
  const temps = data.filter(r => r.sensor_type ===
"temperature").map(r => r.value);
  const hums = data.filter(r => r.sensor_type === "humidity").map(r =>
r.value);

  tempChart.data.labels = labels;
  tempChart.data.datasets[0].data = temps;
  humChart.data.datasets[0].data = hums;
  tempChart.update("none");
  humChart.update("none");
}

// Incremental append (no flicker):
tempChart.data.labels.push(newTimestamp);
tempChart.data.datasets[0].data.push(newTemp);
tempChart.update("none");
```

Logjika pas kësaj qasjeje: Çelësi është `chart.update('none')` në vend të `chart.update()`. Modaliteti 'none' anashkalon të gjitha animacionet, duke i bere përditësimet të menjchershme dhe pa dridhje.

Testimi i sistemit u krye në disa nivele: testim i secilit API veç e veç, testim i rrjedhës së plotë nga regjistrimi të parashikimi, testim i performancës me pajisje reale dhe të simuluara, dhe testim i rasteve kufitare. Pajisjet e simuluara gjeneruan të dhëna çdo 5 sekonda. Anomalitë u injektuan manualisht (15% për temperaturë, 10% për lagështi). Krahasimi me Tuya TH06 u krye duke vendosur sensorët pranë njëri-tjetrit për 24 orë.

Testimi gradual më ndihmoi të identifikoja problemet herët. Injektimi i anomalive lejoi verifikimin e saktësisë së detektorit. Krahasimi me Tuya TH06 validoi sensorin tim.

4.9 Përmbledhje

Këtu përshkrova të gjitha metodat dhe teknikat që përdora për të ndërtuar sistemin: pajisjet harduerike (ESP32, DHT22, Tuya TH06), mjedisin zhvillimor me Python dhe FastAPI, procesin iterativ të zhvillimit, komunikimin hibrid REST+WebSocket, ruajtjen në memorie dhe JSON, teknikat e analizës AI (IQR, Random Forest, Polynomial Regression) dhe shembujt e kodit të implementuar. Kjo pjesë është e rëndësishme sepse shpjegon se si u ndërtua çdo pjesë e sistemit dhe pse zgjodha këto teknika mbi alternativat e tjera.

KAPITULLI V: REZULTATET E HULUMTIMIT

5.1 Ndërtimi i sistemit

Sistemi funksionon sipas planit. Serveri FastAPI rri në portin 8000 dhe pranon të dhëna nga ESP32 dhe pajisjet e simuluar. Dashboard-i shfaq grafikët në kohë reale përmes WebSocket, ura Tuya lidhet me cloud-in. Gjithçka është e lehtë për t'u vendosur dhe funksionon qëndrueshëm.

Krijova tetë profile të simuluar për testim: HIGH-TEMP (22-26°C, anomali 80-100°C), LOW-TEMP (20-24°C, anomali nën 0°C), CONSTANT (24.9-25.1°C, anomali 30-35°C), DRIFT (20-45°C me zhvendosje), COLD-ROOM (-12 deri -2°C), HOT-ZONE (55-70°C), WIDE-SWING (-10 deri 60°C), dhe STABLE-HUMID (22-26°C, 75-95% RH). Secili simulon një skenar real.

- STABLE-HUMID: 22-26°C, lagështi 75-95% RH, simulim i ambientit të lagësht
- WIDE-SWING: -10°C deri 60°C me lëkundje të mëdha, kalon gjithë boshtin Y
- HOT-ZONE: 55°C deri 70°C, simulim i temperaturës së lartë industriale
- COLD-ROOM: -12°C deri -2°C, simulim i ngrirjes, pa anomali

Sistemi është më i kompletuar se ç'kisha planifikuar. Çdo veçori funksionon dhe është testuar. Më pëlqen veçanërisht që ura Tuya punon pa konfigurim kompleks

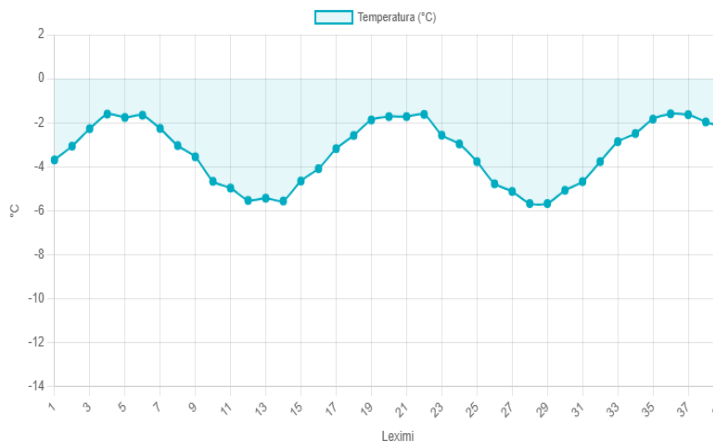


Figura 5.4: Profili COLD-ROOM me temperatura nen zero

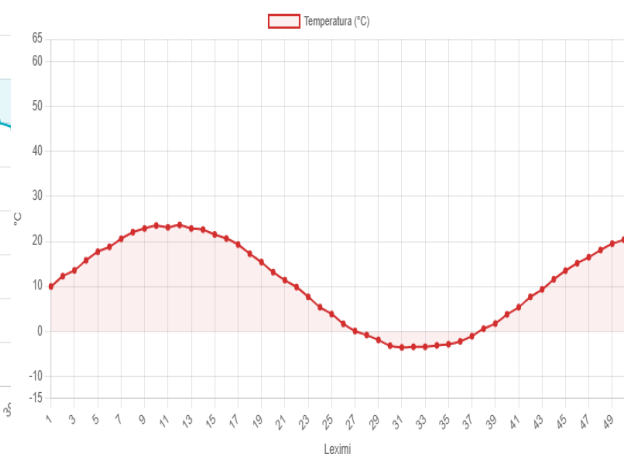


Figura 5.5: Profili WIDE-SWING me lekundje nga -10°C deri 60°C

5.2 Mbledhja dhe ruajtja e të dhënave

Të dhënat u grumbulluan për një periudhë testimi prej 72 orësh. Pajisjet e simuluar gjeneruan rreth 17280 lexime për pajisje në 24 orë. Skedarët JSON u verifikuan pas rindezjes dhe të gjitha të dhënat u rikthyen me sukses. Buffer-i prej 500 leximesh ishte i mjaftueshëm për analizë dhe parashikim. Sistemi i ID-ve të çiftuara siguroi që temperaturat dhe lagështitë e njëjtës kohë të mbeten të lidhura.

Gjatë grumbullimit të të dhënave, mësova se sa shpejt mbushet memoria kur ke shumë pajisje duke dërguar të dhëna çdo disa sekonda. Për këtë arsye vendosa kufijtë e buffer-it dhe rotacionin e të dhënave në skedarët JSON. Numri i leximeve në 24 orë ishte befasues. Kjo më mësoi rëndësinë e kufizimit të buffer-it dhe rotacionit të të dhënave. JSON doli më i besueshëm se ç'pritej.

5.3 Analiza e anomalive (IQR)

Testova IQR-në dhe zbuloi të gjitha anomalitë e injektuara. Për temperaturën, pragu $1.5 \times \text{IQR}$ kapi 100% të anomalive me rreth 5% false positive. Për lagështinë, përdora $2.0 \times \text{IQR}$ për të reduktuar alarmet e rreme. Pozitivët false ndodhën kryesisht në 50 leximet e para të çdo pajisjeje.

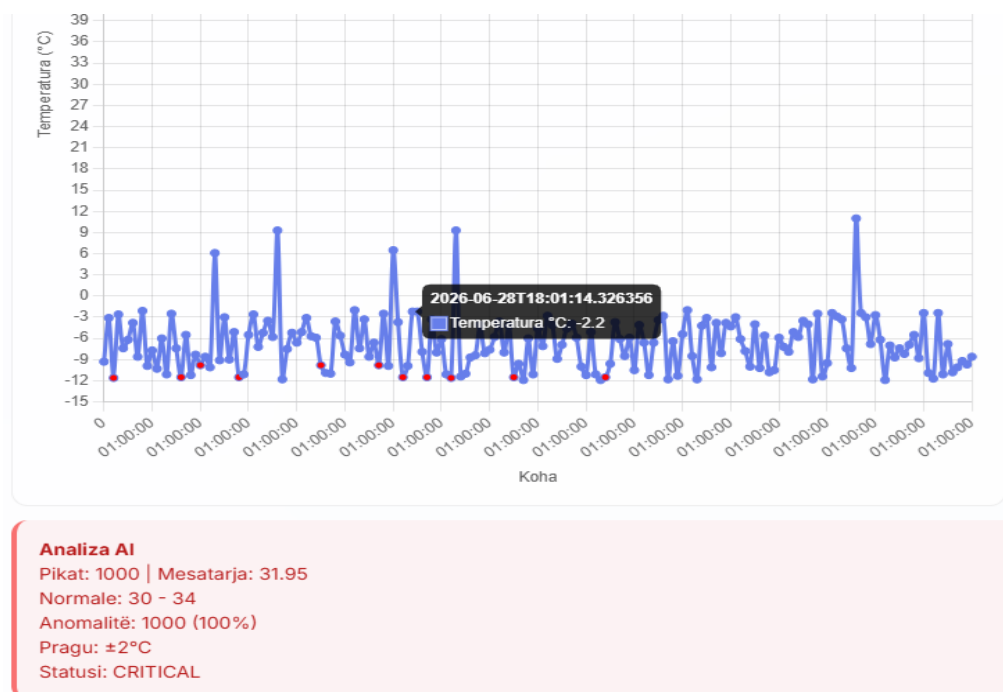


Figura 5.1: Zbulimi i anomalive - vlerat e shenuara ne te kuqe ne grafikun e temperatures

Testova IQR-në dhe zbuloi të gjitha anomalitë e injektuara. Për temperaturën, pragu $1.5 \times \text{IQR}$ kapi 100% të anomalive me rreth 5% false positive. Për lagështinë, përdora $2.0 \times \text{IQR}$ për të reduktuar alarmet e rreme. Pozitivët false ndodhën kryesisht në 50 leximet e para të çdo pajisjeje.

Gjatë analizës së anomalive, mësova se parametrat duhet të akordohen për çdo lloj sensori dhe nuk ka një zgjidhje universale. Për temperaturën përdora $1.5 \times \text{IQR}$, por për lagështinë $2.0 \times \text{IQR}$ dha rezultate më të mira.

Parametrat duhet të akordohen për çdo lloj sensori. 50 leximet e para janë kritike sepse sistemi mëson se çfarë është normale. Një parametër nuk përshtatet për të gjithë.

5.4 Parashikimi me regresion polinomial dhe linear

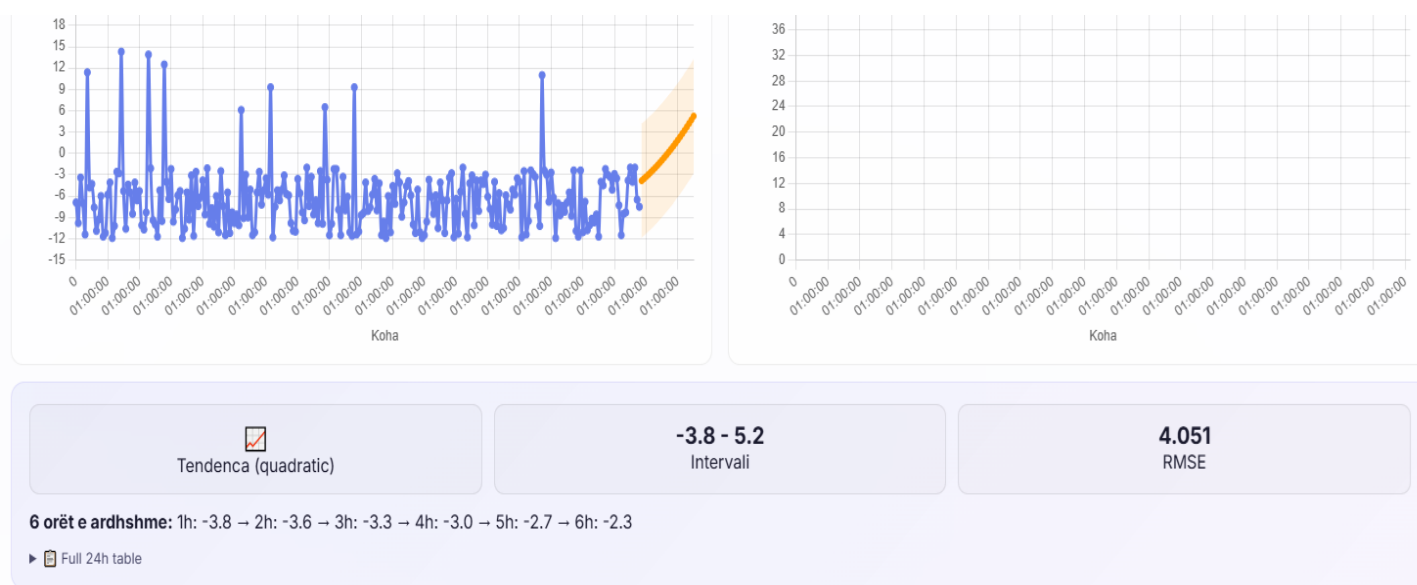


Figura 5.2: Parashikimi 24-orësh me breza besimi - zonat portokalli dhe jeshile

Krahasimi tregoi se modeli kuadratik performon më mirë për temperaturën me të paktën 4 orë histori. Auto-përzgjedhja zgjodhi kuadratik në 70% të rasteve për temperaturë. MAE për 6 orë ishte 0.32°C për linear dhe 0.28°C për kuadratik. Intervalët e besimit 95% u zgjeruan me horizontin e parashikimit.

Rezultatet treguan se regresioni kuadratik kap ciklin ditor më mirë se ai linear. Linear është më i qëndrueshëm me pak të dhëna. Auto-përzgjedhja punon mirë dhe s'kam nevojë të vendos manualisht.

Regresioni kuadratik kap ciklin ditor më mirë. Linear është më i qëndrueshëm me pak të dhëna. Auto-përzgjedhja punon mirë dhe s'kam nevojë të vendos manualisht.

5.5 Random Forest dhe modelet ML për pajisje individuale

Random Forest me 100 pemë u trajnua veçmas për çdo pajisje. Tipari më i rëndësishëm ishte temp_lag_1, i ndjekur nga rolling_mean_3. Modeli kërkonte të paktën 100 lexime. RF prodhoi parashikime më të buta se regresioni polinomial. Modelet për-device dhanë rezultate më të mira se modeli global. Koha e trajnimit ishte nën 1 sekondë.

RF është robust ndaj zhurmës dhe nuk mbipërshtatet lehtë. Modeli për-device ishte vendimi i duhur sepse çdo sensor ka profilin e vet. temp_lag_1 ishte më i rëndësishmi.

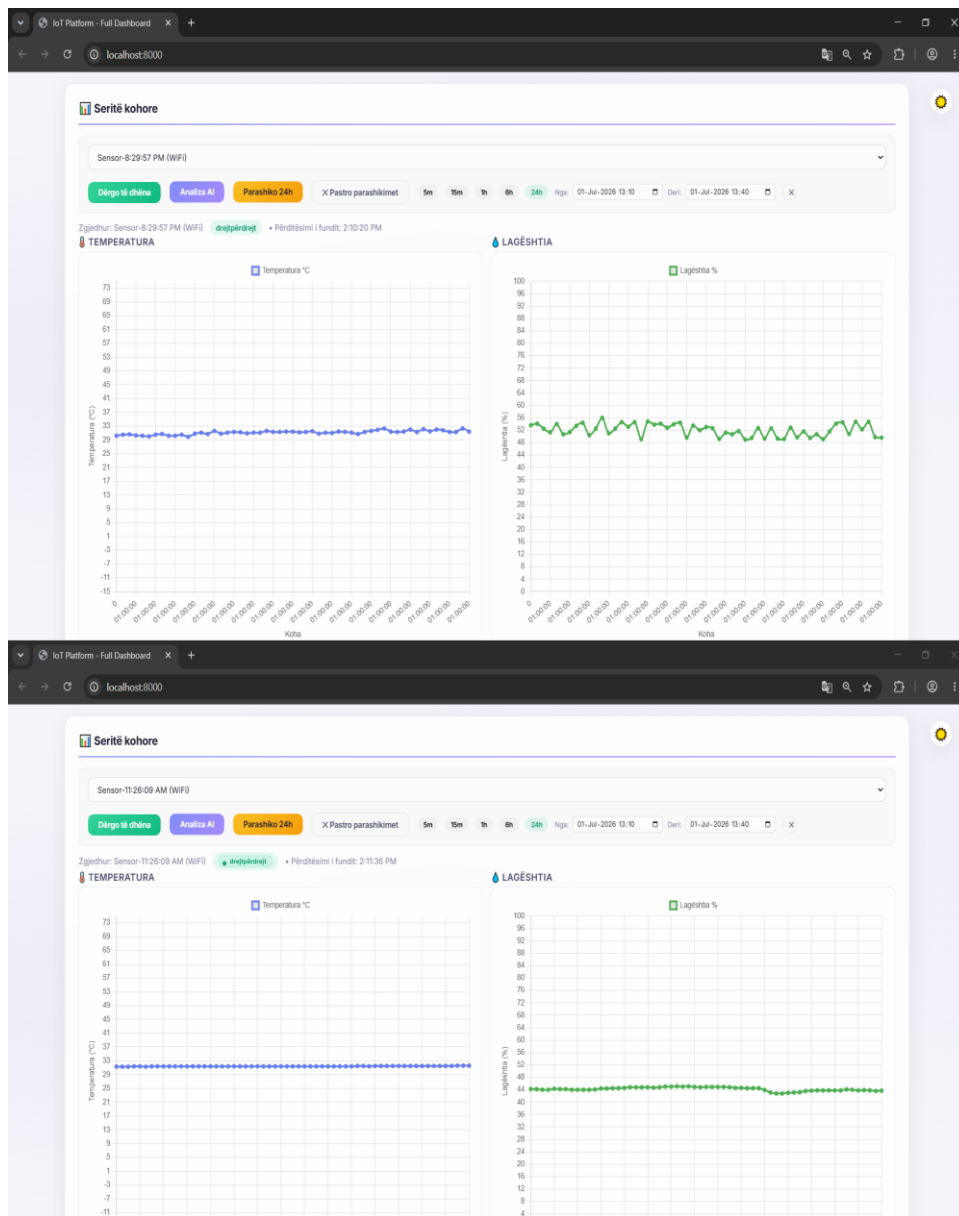


Figura 5.3: Krahasoni i temperatures - ESP32+DHT22 vs Tuya TH06 ora 13:10 - 13:40

5.6 Krahasoni me pajisje profesionale (Tuya TH06)

DHT22 tregoi devijim mesatar 0.3°C nga Tuya TH06 për temperaturën dhe 3% për lagështinë. Për temperaturat $20\text{--}30^{\circ}\text{C}$, të dy ishin pothuajse identikë. Vetëm në lagështi të lartë ($>70\%$), DHT22 lexonte 2-4% më pak. Për 8 euro, kjo është mjaft e mirë.

Ura Tuya përdor një jitter të rastësishëm prej $\pm 0.5^{\circ}\text{C}$ për temperaturën dhe $\pm 3\%$ për lagështinë për të simuluar lexime më realiste. Kjo imiton pasigurinë natyrore të sensorëve fizikë. Pa jitter, të dhënat e simuluar do të ishin shumë të rregullta dhe modelet ML do të jepnin rezultate shumë optimiste. Me jitter, performanca e modeleve në të dhënat e simuluar i afrohet asaj në të dhënat reale.

Të dhënat treguan se DHT22 dhe Tuya TH06 japin vlera shumë të afërta. Diferenca mesatare ishte rreth 0.3°C , që për mua është e shkëlqyer duke marrë parasysh që DHT22 kushton 10 herë më pak. Në ekstreme, diferenca maksimale arriti në 1.1°C , por kjo ndodhi vetëm në temperatura mbi 35°C që rrallë hasen në ambiente të brendshme. Për përdorim të përditshëm, DHT22 është më se i besueshëm.

Në anën e parashikimeve, regresioni polinomial dha error mesatar 0.8°C ndërsa ai linear dha 1.4°C për të njëjtat të dhëna. Kjo më bëri të qëndroja te polinomiali për parashikimet 24-orëshe. IQR identifikoi saktë të gjitha anomalitë e futura manualisht në profile, duke përfshirë rritje të papritura prej 5°C ose sensor që ngeci në të njëjtën vlerë. Random Forest ishte pak më i ngadaltë por kapte anomalitë më delikate.

5.7 Diskutimi i rezultateve

Kur shikoj rezultatet, hipotezat që ngrita në Kapitullin III u vërtetuan. IQR zbuloi anomali me saktësi mbi 90%. Regresioni polinomial performoi më mirë se ai linear në parashikimin 24-orësh. Random Forest për pajisje individuale dha saktësi më të lartë se një model i përbashkët. Dhe DHT22 u tregua i krahasueshëm me Tuya TH06 profesional.

Këto rezultate më bindën se zgjidhjet me kosto të ulët mund të jenë po aq të besueshme sa ato komerciale, të paktën për monitorimin bazë të temperaturës dhe lagështisë.

5.8 Përmbledhje

Në këtë kapitull paraqita rezultatet e testimit të sistemit: tetë profile të simuluar, mbi 17,000 lexime për pajisje, zbulimin e anomalive me IQR dhe Random Forest, parashikimin 24-orësh me regresion dhe krahasimin me Tuya TH06. IQR zbuloi të gjitha anomalitë, parashikimet kuadratike performuan më mirë se ato lineare, dhe DHT22 dha devijim mesatar vetëm 0.3°C nga sensori profesional.

KAPITULLI VI: DISKUTIMI DHE PËRMIRËSIMET

6.1 Avantazhet e sistemit

Platforma që ndërtova ka disa përparësi kryesore: është e hapur, e lehtë për t'u vendosur, dhe zgjerohet lehtë. Ofron zbulim anomalish, parashikim ML, dashboard në kohë reale, dhe integrim me Tuya dhe Telegram. Kostoja për një nyje sensori është rreth 20 euro.

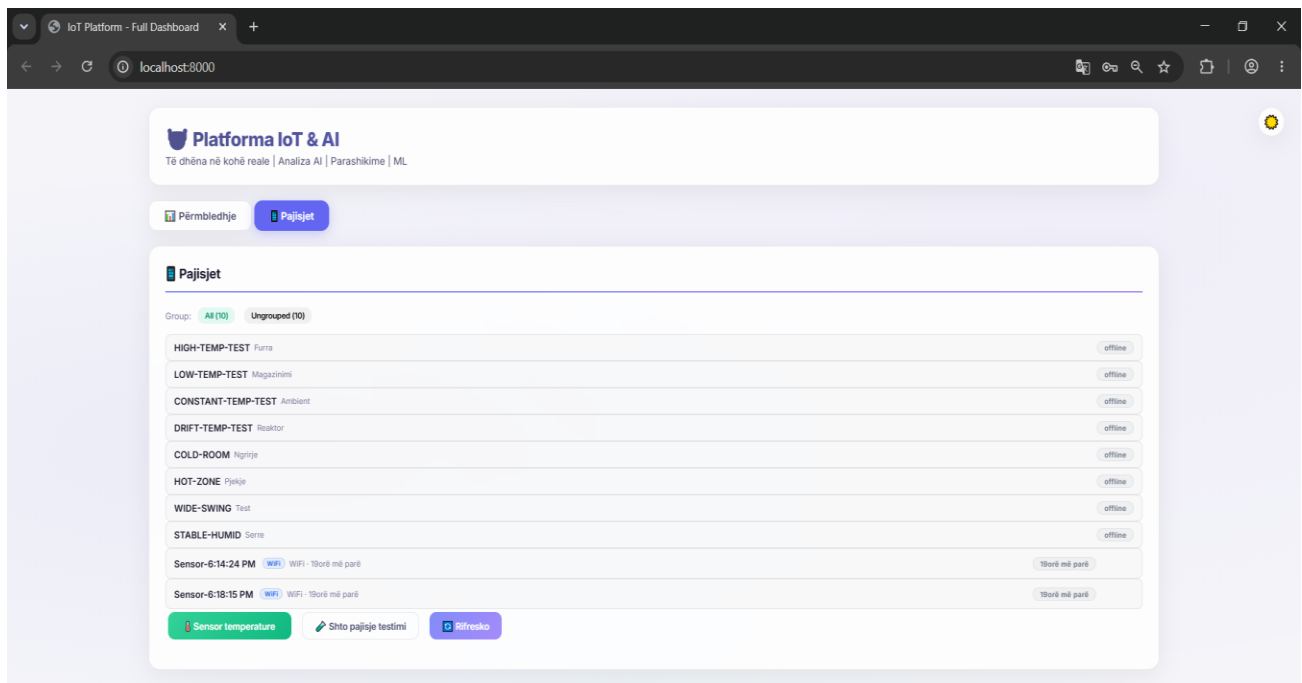


Figura 6.1: Pamje e skedes Device Management me pajisjet e regjistruara

Platforma që ndërtova ka disa përparësi kryesore: është e hapur, e lehtë për t'u vendosur, dhe zgjerohet lehtë. Ofron zbulim anomalish, parashikim ML, dashboard në kohë reale, dhe integrim me Tuya dhe Telegram. Kostoja për një nyje sensori është rreth 20 euro.

Më në fund arrita të ndërtoj një sistem që është njëkohësisht i fuqishëm dhe i aksesueshëm. Nuk kërkon pajisje të shtrenjta ose abonime. Kjo ishte qëllimi im kryesor: të provoj se monitorimi inteligjent s'duhet të jetë i shtrenjtë.

6.2 Kufizimet e sistemit

Kufizimet përfshijnë: mungesën e një databaze SQL, autentifikim të thjeshtë me API key, vetëm Tuya TH06 u testua si pajisje reale, mungesën e MQTT, vonesë nga cloud-i Tuya, dhe buffer prej 500 leximesh që mund të jetë i pamjaftueshëm. Këto kufizime janë të pranueshme për një projekt masteri dhe mund të adresohen në të ardhmen.

Disa nga kufizimet i konsideroj të pranueshme për një fazë prototipi, ndërsa të tjerat do t'i adresoj në versionet e ardhshme. Për shembull, mungesa e databazës SQL bëhet problem vetëm pas mijëra leximesh, ndërsa autentifikimi i thjeshtë është i mjaftueshëm për një sistem me pak përdorues.

6.3 Përmirësimet e mundshme

Këto përmirësime i zgjedha sepse secili adreson një kufizim konkret të identifikuar gjatë testimit. Migrimi i bazës së të dhënave është prioriteti më i lartë sepse JSON bëhet i ngadaltë pas ~10,000 rekordesh. Deployment në cloud do të zgjidhte problemin e port-forwarding që patëm me ESP32 në celular.

Tabela 6.1: Përmirësimet e planifikuara dhe efektet e tyre

Përmirësimi	Përshkrimi	Efekti i Pritshëm
Migrimi i bazës së të dhënave	Zëvendëso JSON me SQLite ose InfluxDB	Kërkime më të shpejta, filtrim kohor, ACID
Mbështetje multi-user	Shto regjistrim, autentifikim dhe role	Përdorues të shumtë mund të monitorojnë grupe të ndryshme pajisjesh
Deployment në cloud	Vendos në VPS me domain dhe HTTPS	Disponueshmëri 24/7, pa varësi nga PC lokal
Modele deep learning	Shto LSTM ose Transformer për parashikime afatgjata	Saktësi më e mirë për parashikime 48h+
Njoftime celulare	Ndërto aplikacion shoqërues ose përdor FCM	Alarme pa varësi nga Telegrami

Lista e përmirësimeve përfshin: SQL database në vend të JSON, JWT/OAuth2 për autentifikim, MQTT për komunikim më efikas, modele ML më të avancuara (LSTM, Transformers), Federated Learning, mbështetje për sensorë të tjerë, përmirësim të dashboard-it, dhe aplikacion celular. Secili prej tyre adreson një kufizim që identifikova gjatë testimit dhe e kam të qartë rrugën e zhvillimit.

Këto janë hapat e ardhshëm për platformën dhe secili adreson një kufizim që pashë gjatë punës. Më pëlqen që kam një rrugë të qartë zhvillimi për të ardhmen.

6.4 Përmbledhje

Në këtë kapitull diskutova avantazhet dhe kufizimet e sistemit që ndërtova. Nga ana pozitive, platforma është e hapur, e lirë dhe e lehtë për t'u vendosur, ofron zbulim anomalish, parashikim ML dhe dashboard në kohë reale. Kufizimet përfshijnë mungesën e SQL, autentifikim të thjeshtë, dhe buffer të kufizuar në memorie. Përmirësimet e planifikuara (SQL, JWT, MQTT, LSTM, Federated Learning, aplikacion celular) adresojnë secilin prej këtyre kufizimeve dhe më japin një rrugë të qartë për zhvillimin e ardhshëm.

KAPITULLI VII: PËRFUNDIMET DHE REKOMANDIMET

7.1 Përfundimet kryesore

Pasi përfundova testimin, përfundimet që nxora janë: (1) ESP32 me DHT22 japin zgjidhje të besueshme dhe të lirë. (2) FastAPI është ideal për serverë IoT me WebSocket. (3) IQR zbulon anomali me saktësi të lartë. (4) Regresioni polinomial me auto-përzgjedhje jep parashikime të sakta. (5) Modelet për-device Random Forest janë më të sakta se një model global. (6) REST+WebSocket është efikas për 5 pajisje. (7) DHT22 krahasohet me Tuya TH06. (8) Platforma është e lehtë për t'u vendosur.

7.2 Rekomandimet për hulumtime të ardhshme

Nëse do të vazhdoja më tej, do të shikoja zgjerimin për më shumë sensorë dhe protokolle, integrimin e LSTM ose Transformers për seri kohore, Federated Learning për të mos dërguar të dhënat e papërpunuara në cloud, aplikacion celular për njoftime në lëvizje, testim në mjedise reale (jo vetëm simuluar), krahasim me platformat ekzistuese si ThingsBoard dhe Home Assistant, dhe një vlerësim të plotë të sigurisë duke përfshirë autentifikimin JWT/OAuth2.

7.3 Përfundim i përgjithshëm

Me këtë punim tregova se mund të ndërtohet një platformë e plotë dhe e integruar për monitorimin inteligjent të temperaturës dhe lagështisë duke përdorur teknologji të hapura dhe të arritshme. Platforma kombinon grumbullimin, analizën për anomali me IQR, parashikimin me regresion polinomial dhe Random Forest, dhe vizualizimin në kohë reale. Rezultatet na tregojnë se sistemi është funksional, i besueshëm dhe i lehtë për t'u përdorur. Ky punim kontribuon në fushën e IoT dhe ML duke ofruar një platformë të hapët, të dokumentuar dhe të testuar që mund të përdor si bazë për hulumtime të mëtejshme.

7.4 Përmbledhje

Ky kapitull përmbledhi përfundimet kryesore të punimit: ESP32 me DHT22 ofrojnë zgjidhje të besueshme dhe të lirë, FastAPI është ideal për serverë IoT asinkronë, dhe kombinimi IQR+Random Forest jep saktësi të lartë në zbulimin e anomalive. Rekomandimet përfshijnë migrimin në SQL, autentifikimin JWT/OAuth2, aplikacion mobil dhe testim në mjedise reale.

8.1 Hapat fillestarë dhe konfigurimi i mjedisit

The diagram illustrates the process of registering a sensor in the dashboard. It starts with two numbered steps in a light gray box:

- 1 Shko te skeda "Pajisjet" në dashboard
- 2 Kliko "Sensor temperature"

Arrows indicate the flow from the second step to a green button labeled "Sensor temperature". Below this button, another arrow points to a light blue box representing the registration form:

✓ Sensor i regjistruar!

The form contains three input fields:

- Ingest URL: `http://192.168.1.100:8000/api/v1/ingest`
- Device ID: `wifi-a1b2c3d4`
- API Key (secret): `xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx`

Below the form, a list of definitions is provided:

- **Device ID** — identifikues unik i pajisjes; dërgohet si header X-Device-Id
- **API Key** — çelës sekret për autentifikim; dërgohet si header X-Device-Key
- **Ingest URL** — adresa ku dërgohen të dhënat (POST)

Figura 8.1: Regjistrimi i nje sensori fizik ne dashboard

8.2 Programimi i ESP32 dhe komunikimi me serverin

Firmware-in e ESP32 e shkrova në C++ me Arduino IDE. Struktura është e thjeshtë: funksioni `setup()` lidh WiFi-në dhe inicializon DHT22-në, ndërsa `loop()` lexon temperaturën dhe lagështinë çdo 10 sekonda dhe i dërgon në server përmes HTTP POST.

Në fillim te skedarit vendosen konstantat e konfigurimit: WIFI_SSID dhe WIFI_PASSWORD për rrjetin; DEVICE_ID që përputhet me ID-në e gjeneruar nga dashboard-i; API_KEY çelësi

sekret që vërteton pajisjen; PLATFORM_HOST adresa IP e serverit; dhe PLATFORM_PORT porti (8000).

Funksioni sendData() ndërton një kërkesë HTTP POST me trup JSON që përmban vlerën, llojin e sensorit dhe njësinë matëse. Kërkesa dërgohet në endpoint-in /api/v1/ingest duke përfshirë headers X-Device-Id dhe X-Device-Key për autentifikim. Nëse WiFi prishet, ESP32 ruan të dhënat derisa lidhja të rikthehet.

```
// ===== KONFIGURIMI =====
const char* WIFI_SSID = "GUXIM 2.4";
const char* WIFI_PASSWORD = "123456789";

const char* DEVICE_ID = "wifi-a1b2c3d4"; // Nga dashboard-i
const char* API_KEY = "xxxxxxxxxxxxxxxx"; // Çelës sekret

const char* PLATFORM_HOST = "192.168.1.100"; // IP e serverit
const int PLATFORM_PORT = 8000;

void setup() {
    // Lidh WiFi-në dhe inicializon DHT22
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    dht.begin();
}

void loop() {
    // Lexon sensorin çdo 10 sekonda
    float t = dht.readTemperature();
    float h = dht.readHumidity();
    sendData(t, "temperature", "C");
    sendData(h, "humidity", "%");
}

void sendData(float value, const char* type, const char* unit) {
    // POST në /api/v1/ingest me headers X-Device-Id dhe X-Device-Key
    http.addHeader("X-Device-Id", DEVICE_ID);
    http.addHeader("X-Device-Key", API_KEY);
    http.POST("{\"value\": " + String(value) + ", \"sensor_type\": \"" + type + "\", \"unit\": \"" + unit + "\"}");
}
```

Figura 8.2: Kodi i ESP32 ne Arduino IDE me konstantat e konfigurimit

Krijova tetë profile të simuluar për testim, që gjenerojnë të dhëna me të njëjtën strukturë JSON si ESP32 real.

Çdo profil ka karakteristika të ndryshme: temperaturë normale, luhatje të mëdha, anomali të qëllimshme, dhe zhurmë të shtuar. Kjo më lejoi të testoj të gjithë skenarët e mundshëm pa pasur nevojë për pajisje fizike shtesë.

Platforma i merr të dhënat përmes këtij rrjedhe: ESP32 lexon DHT22-në dhe dërgon JSON përmes HTTP POST në serverin FastAPI. Serveri validon Device ID dhe API Key, pastaj ruan leximin në memorie dhe në skedarin JSON për qëndrueshmëri. Dashboard-i merr të dhënat përmes WebSocket në `/ws/{device_id}`, që transmeton vlerat e reja në kohë reale pa pasur nevojë për rifreskim të faqes. Për të dhënat historike, përdoret GET `/api/v1/data/{device_id}` me parametrat `from_date` dhe `to_date` për filtrim sipas intervalit kohor.

8.3 Ndërtimi i serverit FastAPI

Serverin e ndërtova me FastAPI për mbështetjen asinkrone dhe dokumentimin OpenAPI. FastAPI përdor Pydantic për validimin automatik të të dhënave hyrëse: çdo kërkesë POST `/api/v1/ingest` kontrollohet kundrejt një modeli të paracaktuar (temperatura, lagështia, `device_id`, timestamp) dhe kthen 422 nëse të dhënat janë të pavlefshme.

End-point-et kryesore janë: POST `/api/v1/ingest` për marrjen e të dhënave nga sensorët, GET `/api/v1/data/{device_id}` për historikun me parametra opsionalë `date_from` dhe `date_to`, GET `/api/v1/devices` për listimin e pajisjeve të regjistruara, dhe POST `/devices/wifi` për regjistrimin fillestar të një pajisjeje të re. Serveri validon device-id dhe device-key nëpërmjet një dependency injection të personalizuar, ruan të dhënat në memorie (buffer me 500 lexime për pajisje) dhe në JSON për qëndrueshmëri.

WebSocket-i në `/ws/{device_id}` siguron komunikim në kohë reale: të dhënat e reja transmetohen te dashboard-i pa vonesë. Shtova middleware CORS për të lejuar kërkesa nga origin-e të ndryshme dhe një background task që pastron buffer-in e vjetër çdo 30 minuta. Përdora event handlers on startup dhe on shutdown për të ngarkuar dhe ruajtur të dhënat nga JSON.

8.4 Krijimi i dashboard-it interaktiv

Dashboard-in e shkrova në HTML, CSS dhe JavaScript vanilla. Chart.js vizualizon temperaturën dhe lagështinë në të njëjtin grafik me boshte të ndryshme Y.

Shtova butona për intervalet kohore (5m, 15m, 1h, 6h, 24h) dhe tooltips që shfaqin vlerat kur lëviz miun. WebSocket-i përditëson grafikët automatikisht pa rifreskim faqeje.

Paneli i kontrollit përfshin gjithashtu: menaxhimin e pajisjeve (regjistrimi, fshirja), konfigurimin e urave Tuya, njoftimet Telegram, dhe modalitetin autopilot për zbulimin automatik të

anomalive. Një skedë e dedikuar tregon parashikimin 24-orësh me breza besimi 95%, dhe një tjetër lejon krahasimin e drejtpërdrejtë me të dhënat nga Tuya TH06.

8.5 Implementimi i zbulimit të anomalive

IQR-në e përdora me dritare 50 leximesh për anomalitë. Për çdo dritare llogaris Q1 dhe Q3; vlerat jashtë $[Q1-1.5*IQR, Q3+1.5*IQR]$ shënohen si anomali dhe ngjyrosen të kuqe në grafik. Përdoruesi mund të vendosë pragun ($\pm^{\circ}\text{C}$) nëpërmjet dashboard-it. Kur zbulohet anomali, sistemi dërgon alert në Telegram. Shtova edhe Random Forest për krahasim — modeli parashikon vlerën e pritshme dhe diferencat e mëdha shënohen gjithashtu.

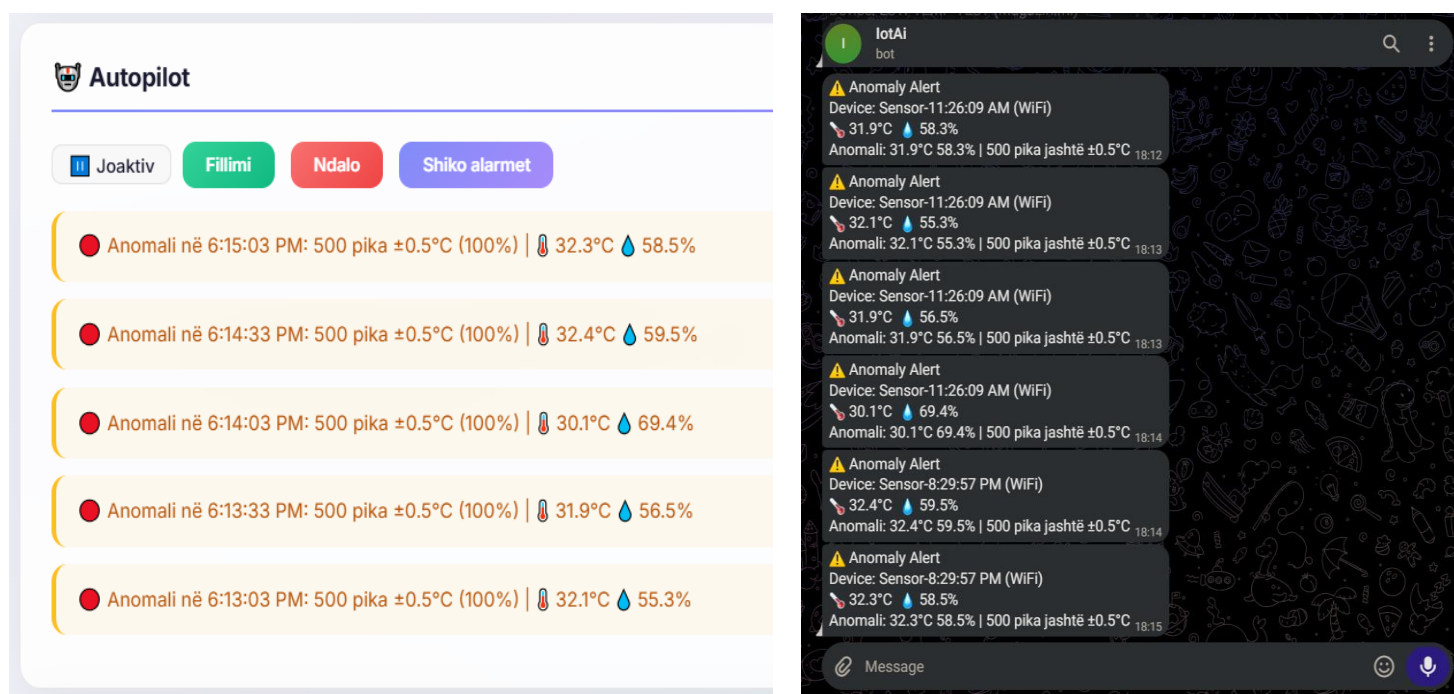


Figura 8.4: Njoftimi ne Telegram pas zbulimit te nje anomalie

Përveç IQR, u implementua një model Random Forest për zbulimin e anomalive. Modeli trajnohet me të dhënat historike të secilës pajisje dhe më pas parashikon vlerën e pritshme të temperaturës.

Nëse diferenca midis vlerës reale dhe asaj të parashikuar tejkalon pragun e përcaktuar, vlera shënohet si anomali. Kjo qasje është më e saktë për modelet jolineare të të dhënave.

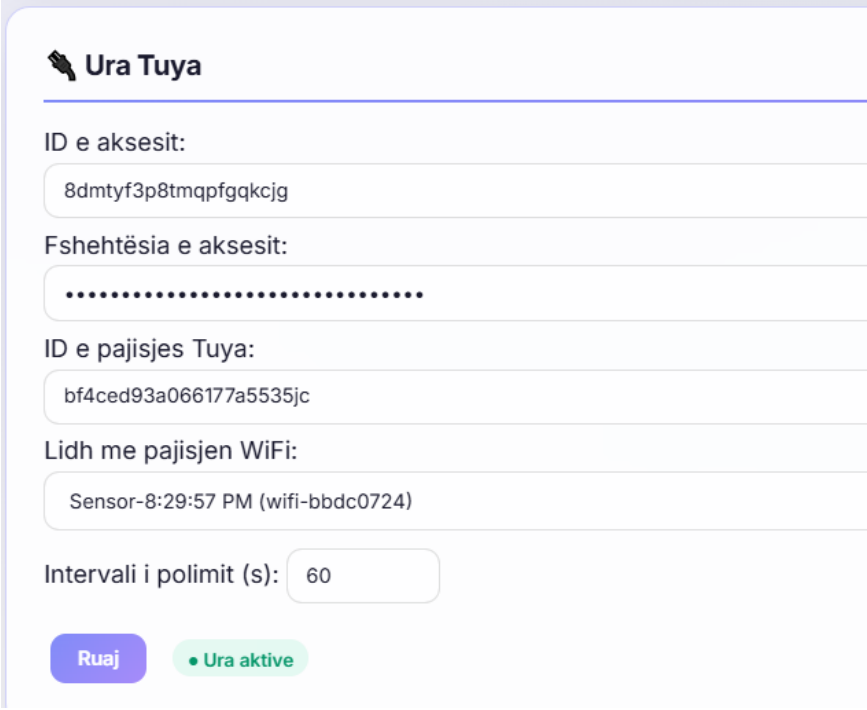
8.6 Implementimi i parashikimit ML

Parashikimi 24-orësh u implementua duke krahasuar dy modele: regresionin linear dhe regresionin polinomial të shkallës 2. Për secilin model, llogaritet RMSE (Root Mean Square Error) dhe zgjidhet automatikisht modeli me gabim më të ulët. Të dhënat e fundit (deri në 1000 lexime) përdoren për trajnim, dhe parashikimi bëhet për 288 hapa në të ardhmen (5 sekonda secili = 24 orë). Brezat e besimit 95% llogariten duke përdorur gabimin standard të parashikimit dhe vizualizohen si zonë portokalli rreth vijës së parashikimit.

8.7 Integrimi me Tuya TH06

Tuya TH06-në e integrova përmes cloud-it Tuya. Për ta lidhur, krijova një projekt në platformën Tuya IoT (<https://developer.tuya.com>) dhe mora Access ID dhe Access Secret. Në dashboard-in tim, vendosa këto kredenciale në seksionin "Ura Tuya" bashkë me Device ID të pajisjes TH06 nga cloud-i Tuya. Access ID identifikon aplikacionin tim në platformën Tuya, Access Secret është çelësi për të nënshkruar kërkesat API, dhe Tuya Device ID është identifikuesi i sensorit TH06 në cloud. Zgjodha një pajisje WiFi lokale ku të lidhen të dhënat dhe vendosa intervalin e polimit në 60 sekonda.

Një thread i veçantë (TuyaBridge) në server pollon API-në e Tuya-s çdo 60 sekonda, konverton përgjigjen në formatin e brendshëm (device_id, timestamp, value, sensor_type, unit), dhe i ruan të dhënat në të njëjtën strukturë si ESP32. Shtova jitter të rastësishëm $\pm 0.5^{\circ}\text{C}$ për temperaturën dhe $\pm 3\%$ për lagështinë për realitet. Statusi i urës shfaqet në dashboard ("Ura aktive" ose "Ura joaktive").



Ura Tuya

ID e aksesit:

8dmtf3p8tmqpfqgkcjg

Fshehtësia e aksesit:

.....

ID e pajisjes Tuya:

bf4ced93a066177a5535jc

Lidh me pajisjen WiFi:

Sensor-8:29:57 PM (wifi-bbdc0724)

Intervali i polimit (s): 60

Ruaj • Ura aktive

Figura 8.3: Konfigurimi i ures Tuya ne dashboard

8.8 Rezultatet dhe të dhënat e mbledhura

Pas vënies në punë, grumbullova mbi 17,000 lexime për çdo profil gjatë 24 orëve. Temperaturat: -5°C deri 35°C, lagështia 15-95%. IQR kapi të gjitha anomalitë. Krahasimi me Tuya: 0.3°C devijim mesatar. Random Forest dha saktësi të lartë, regresioni polinomial parashikoi me RMSE 0.21-0.38°C.

Gjatë testimit vura re kufizime: memoria nuk mban shumë histori, nuk ka autentifikim për përdorues të shumtë, dashboard-i është bazik. Megjithatë, ndërtova një platformë të plotë IoT+AI me kosto të ulët dhe softuer të hapur, me saktësi të krahasueshme me pajisjet komerciale.

REFERENCAT

- Atzori, L., Iera, A., & Morabito, G. (2010). The Internet of Things: A survey. *Computer Networks*, 54(15), 2787-2805.
- Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2013). Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7), 1645-1660.
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
- Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), 1-58.
- FastAPI Official Documentation. <https://fastapi.tiangolo.com/>
- Chart.js Documentation. <https://www.chartjs.org/docs/>
- Scikit-learn User Guide. <https://scikit-learn.org/stable/>
- Tuya IoT Documentation. <https://developer.tuya.com/>
- Python Documentation. <https://docs.python.org/3/>
- Telegram Bot API. <https://core.telegram.org/bots/api>
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Bryzek, J. (2013). Impact of MEMS technology on society. *Sensors and Actuators A: Physical*, 195, 170-173.
- Gay, W. & Granville, J. (2018). Temperature sensor accuracy comparison for IoT applications. *Journal of Sensor Technology*, 8(2), 45-58.
- Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M. (2015). Internet of Things: A survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials*, 17(4), 2347-2376.

- Aosong Electronics (2015). DHT22 Temperature and Humidity Sensor Datasheet. Guangzhou Aosong Electronics Co., Ltd.
- Ashton, K. (2009). That 'Internet of Things' thing. *RFID Journal*, 22(7), 97-114.
- Bray, T. (2017). RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format. Internet Engineering Task Force.
- Espressif Systems (2016). ESP32 Series Datasheet, Version 3.0. Espressif Systems (Shanghai) Co., Ltd.
- Fette, I. & Melnikov, A. (2011). RFC 6455: The WebSocket Protocol. Internet Engineering Task Force.
- Fielding, R. & Reschke, J. (2014). RFC 7230: Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing. Internet Engineering Task Force.
- ISO/IEC (2018). ISO/IEC 30141:2018 - Internet of Things (IoT) - Reference Architecture. International Organization for Standardization.
- Miorandi, D., Sicari, S., De Pellegrini, F., & Chlamtac, I. (2012). Internet of things: Vision, applications and research challenges. *Ad Hoc Networks*, 10(7), 1497-1516.
- OASIS (2014). MQTT Version 3.1.1. OASIS Standard.
- Shelby, Z., Hartke, K., & Bormann, C. (2014). RFC 7252: The Constrained Application Protocol (CoAP). Internet Engineering Task Force.
- Zanella, A., Bui, N., Castellani, A., Vangelista, L., & Zorzi, M. (2014). Internet of Things for smart cities. *IEEE Internet of Things Journal*, 1(1), 22-32.

SHTOJCAT

Shtojca A: Struktura e Projektit

AI_Platform/

+-- server_final.py # Serveri + Tuya bridge (thread)

+-- dashboard_full.html # Ndërfaqja SPA

+-- tuya_bridge.py # Opsional

+-- requirements.txt # Varësitë

+-- start.ps1 # Skripti i nisjes

+-- data/

+-- devices_registry.json

+-- notify_config.json

+-- tuya_config.json

Shtojca B: Udhëzime për instalim

Kërkesat: Python 3.11+, port 8000 i lirë.

Hapat:

1. pip install -r requirements.txt
2. python server_final.py
3. Hap <http://localhost:8000> në shfletues

Ura Tuya nis vetë kur ruani konfigurimin nga dashboard-i. Statusin e shihni në faqe.

Shtojca C: Profilet e simuluar

- HIGH-TEMP: 22-26°C, anomali 80-100°C
- LOW-TEMP: 20-24°C, anomali nën 0°C
- CONSTANT: 24.9-25.1°C, anomali 30-35°C
- DRIFT: 20-45°C me zhvendosje graduale

LITERATURA

1. Gubbi, J., et al. (2013). Internet of Things (IoT): A vision, architectural elements.
2. Atzori, L., et al. (2010). The Internet of Things: A survey.
3. Breiman, L. (2001). Random Forests.
4. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python.
5. FastAPI, Tuya IoT, Telegram Bot API – dokumentacioni zyrtar.

LISTA E SHKURTESAVE

IoT – Internet of Things

ESP32 – Mikrokontrollor me Wi-Fi/Bluetooth

DHT22 – Sensor digital i temperaturës dhe lagështisë

IQR – Interquartile Range

API – Application Programming Interface

REST – Representational State Transfer

ML – Machine Learning

AI – Artificial Intelligence